FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

P2

Relatório consolidado

arquitetura e atividades do 2º bimestre

Avaliação individual · 2026.2

Luana Beatriz Ribeiro Abrantes

Produto: **GiroRural**
https://github.com/Luanaabrantes/girorural

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 13/11/2026

Sumário

1 Introdução 2
2 Metodologia 2
3 Arquitetura do SaaS 3
3.1 Visão geral e componentes 3
3.2 Onde roda 4
3.3 Fluxo de uma requisição 5
3.4 Fluxo de assinatura 5
3.5 Modelo de dados do domínio 7
3.6 Segurança 8
3.7 Observabilidade, CI/CD e documentação da API 8
3.8 Camadas IaaS, PaaS e SaaS do projeto 9
4 Quadro de entregas 9
5 Atividades realizadas 10
5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe 10
5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve 12
5.3 Atividade 9 — Fork e plano de negócio 15
5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes 18
5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe 23
6 Considerações finais 28
7 Declaração de autoria 29

1 Introdução

No primeiro bimestre de Introdução à Computação em Nuvem (ISW055), o trabalho girou em torno de um catálogo de filmes de Tom Hanks. A cada atividade o catálogo ganhou uma peça nova: o microsserviço de autenticação, o controle de acesso por papel, a auditoria com Redis, o perfil com fotos no MinIO e a publicação no Portainer. No segundo bimestre essas peças deixaram de ser exercícios soltos. Primeiro o catálogo passou a cobrar um plano Premium pelo Stripe e ganhou uma documentação reversa. Depois o histórico inteiro foi copiado para um repositório novo e virou a base de um negócio meu, o GiroRural.

O GiroRural resolve um problema que conheço do campo: muitos produtores têm trator, colheitadeira ou implemento parado boa parte do ano, enquanto outros precisam dessas máquinas por poucos dias e não encontram quem alugue perto. Na plataforma, o proprietário anuncia a máquina por diária, o locatário busca, pede a reserva, paga depois da aprovação e só então recebe o contato do dono para combinar a retirada. Para o proprietário, o produto é um SaaS: toda conta começa no plano Gratuito, com três anúncios novos por mês, e pode assinar o plano Pro por R$ 39,90 mensais pelo Stripe para anunciar sem limite.

Este relatório tem duas metades. A seção 3 descreve a arquitetura do sistema que está no ar em luana-abrantes-isw055.lapps.studio: componentes, onde rodam, como uma requisição e uma assinatura atravessam o sistema, o modelo de dados, a segurança, a observabilidade e as camadas de nuvem. As seções 4 e 5 trazem o quadro das atividades 7 a 11, com a data planejada, a data realizada e a prova de cada entrega, e a seção 6 fecha com o que aprendi e o que ainda falta para o GiroRural ser um produto de verdade.

2 Metodologia

As atividades foram desenvolvidas em Node.js com Express, no Visual Studio Code, com versionamento no GitHub e execução em contêineres Docker. Cada entrega começou numa branch própria (por exemplo `atividade-7-stripe-premium`, `atividade-8-documentacao-reversa`, `atividade-10` e `saas-completo-stripe`) e foi incorporada à `main` depois de testada localmente com o Docker Compose e com a suíte de testes automatizados (`npm test`). A publicação seguiu o mesmo caminho do primeiro bimestre: o GitHub Actions constrói as imagens e as publica no GHCR, e eu atualizo a stack `girorural-luana-abrantes` no Portainer com a nova tag. Nas atividades 10 e 11 usei também um assistente de IA (Claude Code) para parte da implementação e da revisão; esses commits aparecem no histórico com o autor "Claude" e foram revisados e publicados por mim.

Na atividade 9 criei o repositório `Luanaabrantes/girorural` a partir de uma cópia completa do repositório `Luanaabrantes/catalogo-filmes`, preservando todos os commits, branches e datas. Por isso as atividades 7 e 8, feitas ainda no catálogo de filmes, têm o mesmo hash, a mesma data e a mesma hora nos dois repositórios, e os links deste relatório apontam para o repositório novo. O plano de negócio, o README do produto e as issues de desenvolvimento foram escritos nesse repositório, e a arquitetura de microsserviços foi reaproveitada inteira; o que mudou foi o domínio (máquinas, anúncios, reservas e planos).

As datas e horas realizadas saíram de duas fontes verificáveis. Para as atividades 7 e 8, usei a data do commit de entrega no GitHub, conferida com `git log --date=format:"%d/%m/%Y %H:%M"`. Da atividade 9 em diante, usei a hora de abertura da issue de entrega que menciona @siriani, conferida pela API do GitHub, e indico também o commit correspondente. Todas as horas estão no fuso de Brasília (UTC−03); alguns commits feitos com o assistente foram gravados em UTC e foram convertidos. A situação de cada atividade compara essa data com a data planejada da lista de atividades, e a arquitetura da seção 3 foi escrita lendo o código do commit `0aac7bb`, que é o que está no ar.

3 Arquitetura do SaaS

3.1 Visão geral e componentes

O GiroRural é formado por cinco contêineres numa única rede Docker interna (`catalogo-network`), mais três dependências externas. Só o `girorural-api` publica porta (8216 no host, 3000 no contêiner), e é por ele que passa tudo o que vem do domínio, inclusive os webhooks do Stripe. Os demais serviços usam apenas `expose` e só existem dentro da rede da stack.

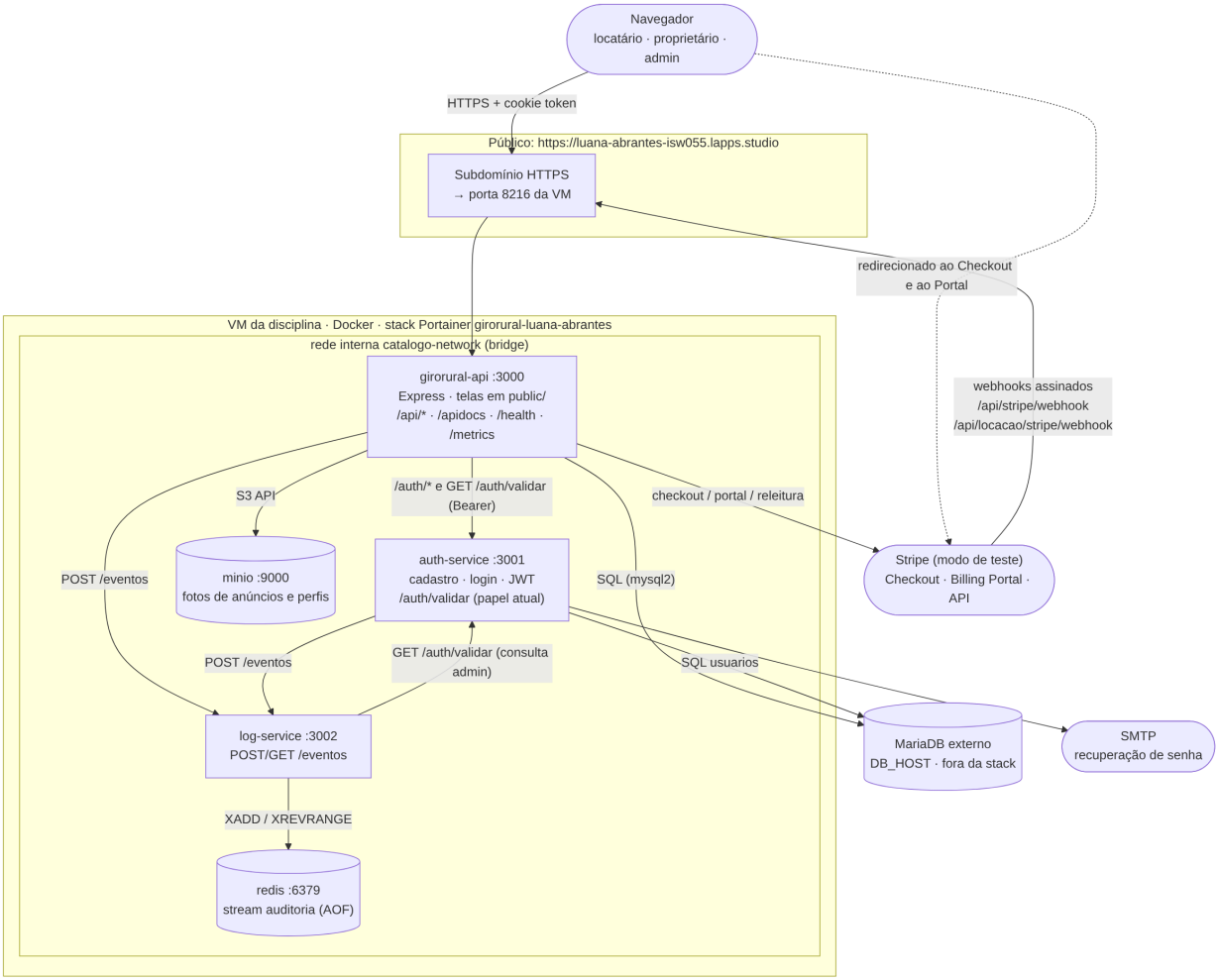


Figura 1: Componentes do GiroRural: o que é público (subdomínio) e o que só existe na rede interna do Docker. Fonte: `docs/p2/diagramas/componentes.mmd`.

Componente	Responsabilidade	Tecnologia / imagem	Público?
app (`girorural-api`)	Telas em `public/`, landing, API `/api/*` de anúncios, reservas, locação, perfil, planos e administração; checkout e portal do Stripe; webhooks; Swagger em `/apidocs`; `/live`, `/health` e `/metrics`.	Node.js + Express · `ghcr.io/luanaabrantes/girorural-api`	sim (domínio, porta 8216)
auth-service	Cadastro, login com JWT (validade de 8 h), recuperação de senha por e-mail, troca de papel	Node.js + Express ·	não (`auth-service:3001`)

Componente	Responsabilidade	Tecnologia / imagem	Público?
	e `GET /auth/validar`, que relê o papel atual no banco a cada requisição (RBAC).	`girorural-auth-service`	
log-service	Recebe eventos de negócio e auditoria (`POST /eventos`) da API e do auth-service; consulta administrativa em `GET /eventos`.	Node.js + Express · `girorural-log-service`	não (`log-service:3002`)
Redis	Stream `auditoria` com persistência AOF no volume `audit-redis-data`.	`redis:7.4-alpine`	não (`redis:6379`)
MariaDB	Usuários, domínio (equipamentos, anúncios, reservas), planos e assinaturas. Externo à stack (`DB_HOST`).	MariaDB da disciplina	não
MinIO	Fotos dos anúncios e de perfil (volume `minio-data`); no banco fica só a chave do objeto.	`girorural-minio`	não (`minio:9000`)
Stripe (externo)	Checkout de assinatura, Billing Portal e checkout de locação; webhooks assinados para `/api/stripe/webhook` e `/api/locacao/stripe/webhook`.	serviço SaaS (modo de teste)	webhook → app
SMTP (externo)	Envio do link de redefinição de senha (`MAIL_*`).	provedor de e-mail	não

3.2 Onde roda

Tudo roda na mesma stack do Portainer das atividades anteriores, `girorural-luana-abrantes`, na VM da disciplina e no mesmo subdomínio (luana-abrantes-isw055.lapps.studio). A stack lê o arquivo `docker-compose.portainer.yml` deste repositório e usa imagens publicadas no GHCR pelo GitHub Actions (`ghcr.io/luanaabrantes/girorural-api`, `girorural-auth-service`, `girorural-log-service` e `girorural-minio`), sempre com a tag `sha-<commit>` definida na variável `IMAGE_TAG`. O Redis usa a imagem oficial `redis:7.4-alpine`.

Contêiner	Porta	Memória	Health check
`girorural-api`	8216 → 3000	`mem_limit: 1g`	`/health` a cada 5 s
`auth-service`	3001 (interna)	`mem_limit: 1g`	`/health` a cada 5 s
`log-service`	3002 (interna)	`mem_limit: 1g`	`/health` a cada 5 s; sobe depois do Redis saudável
`redis`	6379 (interna)	`mem_limit: 1g`	`redis-cli ping`
`minio`	9000 (interna)	`mem_limit: 1g`	`/minio/health/live`

Todos usam `restart: unless-stopped`. Os segredos são variáveis da stack no Portainer e nunca aparecem no repositório; o arquivo `.env.example` lista só os nomes:

- **Banco:** `DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`
- **Sessão:** `JWT_SECRET` · **MinIO:** `MINIO_ACCESS_KEY`, `MINIO_SECRET_KEY`, `MINIO_BUCKET`
- **E-mail:** `MAIL_HOST`, `MAIL_PORT`, `MAIL_USER`, `MAIL_PASS`, `MAIL_FROM`
- **Stripe (teste):** `STRIPE_SECRET_KEY`, `STRIPE_PRICE_ID`, `STRIPE_PRICE_FREE_ID`, `STRIPE_WEBHOOK_SECRET`, `STRIPE_PORTAL_CONFIGURATION_ID`, `STRIPE_LOCACAO_WEBHOOK_SECRET`, `APP_BASE_URL`
- **Catálogo e deploy:** `CATALOGO_INICIAL`, `CATALOGO_PROPRIETARIO_EMAIL`, `CATALOGO_PROPRIETARIO_SENHA`, `PAGAMENTO_SIMULADO`, `IMAGE_TAG`

3.3 Fluxo de uma requisição

A ação escolhida é a mais comum do negócio: um locatário pede a reserva de uma máquina. O navegador envia `POST /api/reservas` com o cookie `token` (httpOnly, SameSite=Lax). A API não decodifica o JWT sozinha: o middleware chama o auth-service, que valida a assinatura e relê o papel atual no banco. Só depois a regra de negócio roda numa transação e o evento vai para a auditoria.

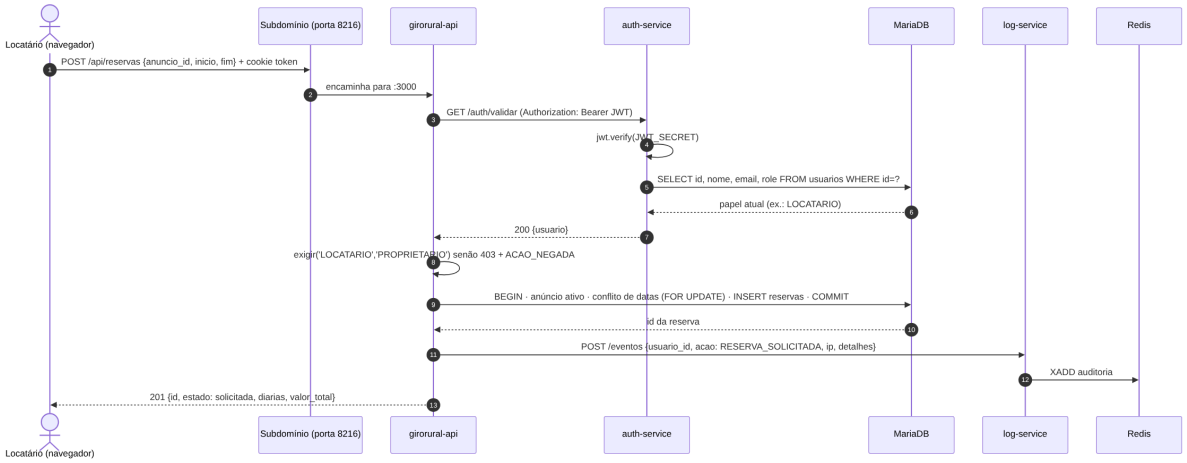


Figura 2: Fluxo de `POST /api/reservas`, do navegador até a resposta. Fonte: `docs/p2/diagramas/requisicao.mmd`.

Se o auth-service não responder em 2 s, a API devolve 503; se o papel não puder reservar, devolve 403 e registra `ACAO_NEGADA` no log-service. A gravação da auditoria é de melhor esforço: uma falha no log-service não desfaz a reserva já confirmada no banco.

3.4 Fluxo de assinatura

O app só abre caminhos (checkout e portal); quem muda o plano no banco é sempre o webhook. Voltar do checkout com `?plano=sucesso` não altera nada.

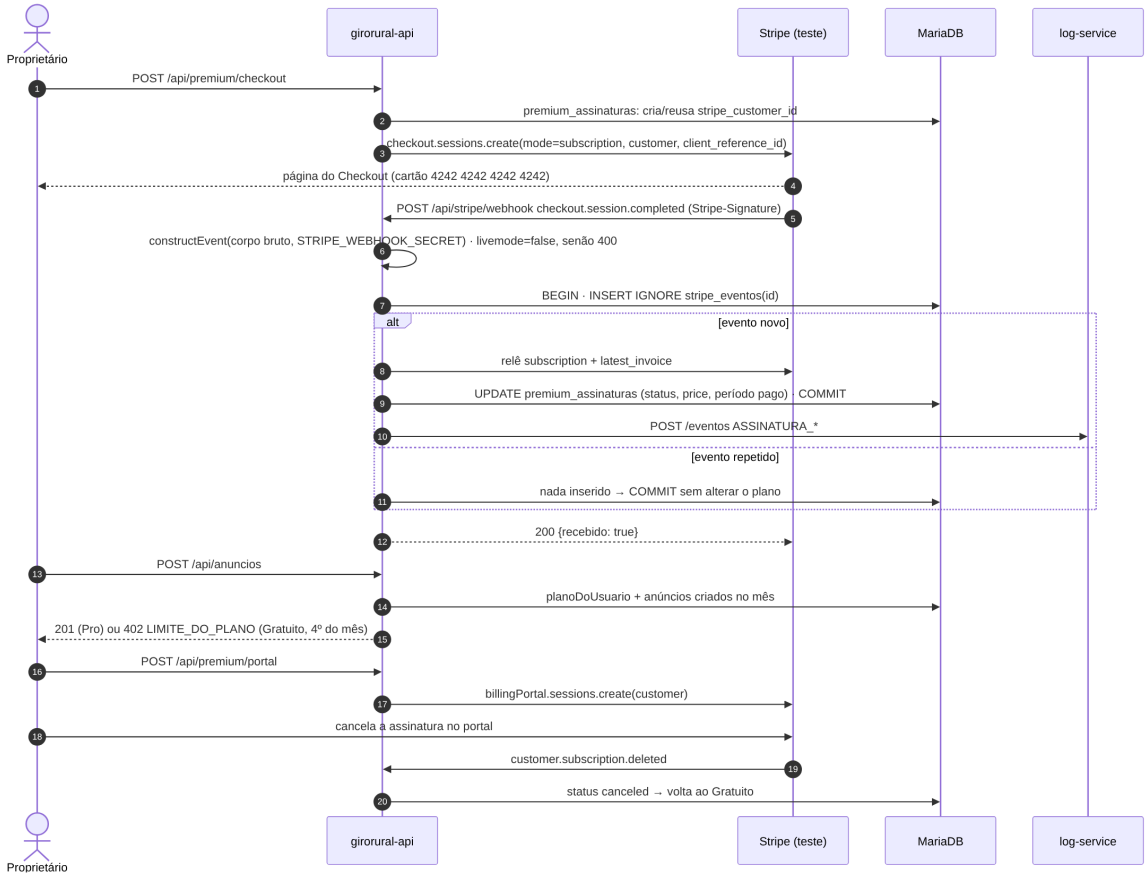


Figura 3: Ciclo da assinatura: checkout, webhook, plano no banco, limite na API e cancelamento pelo portal. Fonte: `docs/p2/diagramas/assinatura.mmd`.

1. **Checkout.** `POST /api/premium/checkout` cria (ou reaproveita) o customer do usuário e abre uma sessão `mode: subscription` com `client_reference_id` igual ao id do usuário. O cartão de teste 4242 é digitado no Stripe e nunca passa pelo servidor.
2. **Validação do webhook.** `POST /api/stripe/webhook` recebe o corpo bruto (`express.raw`) e chama `stripe.webhooks.constructEvent` com `STRIPE_WEBHOOK_SECRET`. Sem assinatura válida, ou com `livemode: true`, responde 400 e não grava nada.
3. **Plano no banco.** O app relê a assinatura e a última fatura no próprio Stripe, confere que o customer e o `client_reference_id` são do usuário e atualiza `premium_assinaturas` (status, preço, período pago). Eventos tratados: `checkout.session.completed`, `customer.subscription.updated`, `customer.subscription.deleted`, `invoice.payment_failed` e `invoice.paid`.
4. **Limite liberado na API.** `POST /api/anuncios` chama `exigirLimiteAnuncios`, que lê o plano no banco a cada requisição. No Gratuito, o 4º anúncio do mês recebe 402 (`LIMITE_DO_PLANO`); no Pro não há limite.
5. **Cancelamento.** `POST /api/premium/portal` abre o Billing Portal para o customer da sessão. Ao cancelar, chega `customer.subscription.deleted`, o status vira `canceled` e a conta volta ao Gratuito.
6. **Evento repetido.** Na mesma transação que atualiza o plano, o id do evento entra em `stripe_eventos` (chave primária) com `INSERT IGNORE`. Se o Stripe reenviar, nada é inserido e o processamento é pulado; se algo falhar, a transação desfaz também o registro do evento e o Stripe pode tentar de novo. A coluna `state_event_created` impede que um evento antigo, chegando fora de ordem, sobrescreva um estado mais novo.

3.5 Modelo de dados do domínio

As tabelas do domínio ficam no mesmo MariaDB da tabela `usuarios`, criada ainda no catálogo de filmes. As migrações estão em `database/` e são aditivas e reexecutáveis.

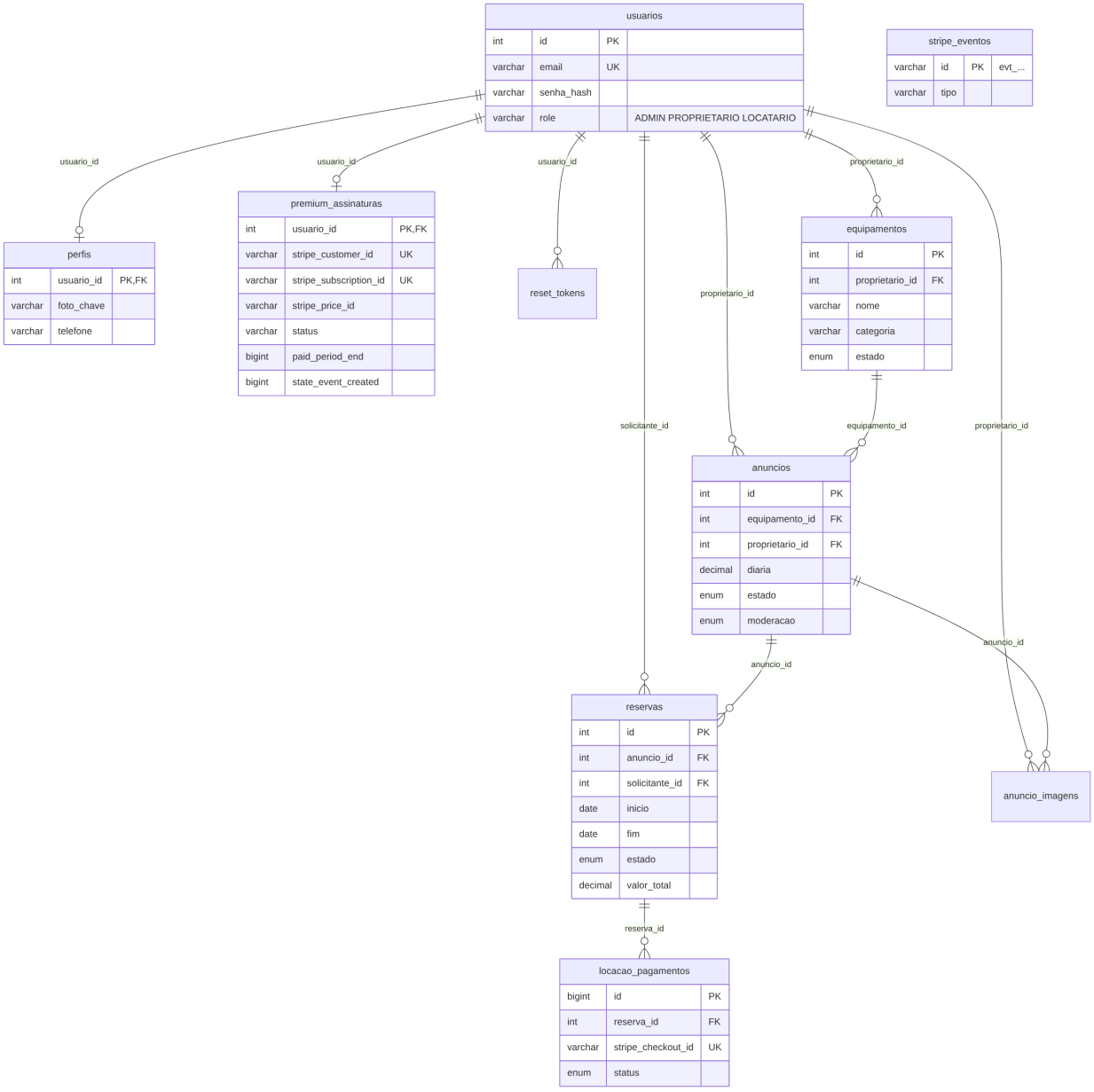


Figura 4: Modelo de dados do domínio e chaves estrangeiras (colunas principais). Fonte: `docs/p2/diagramas/dados.mmd`.

Tabela	Papel e chaves estrangeiras
`usuarios`	Conta e papel (`role`: ADMIN, PROPRIETARIO ou LOCATARIO).
`perfis`	Telefone, cidade, UF e chave da foto no MinIO; `usuario_id` ⟶ `usuarios`.
`equipamentos`	Máquina do proprietário; `proprietario_id` ⟶ `usuarios`.
`anuncios`	Diária, cidade, estado e moderação; `equipamento_id` ⟶ `equipamentos`, `proprietario_id` ⟶ `usuarios`.
`anuncio_imagens`	Galeria de fotos; `anuncio_id` ⟶ `anuncios`, `proprietario_id` ⟶ `usuarios`.

Tabela	Papel e chaves estrangeiras
`reservas`	Período, estado e valor; `anuncio_id` → `anuncios`, `solicitante_id` → `usuarios`.
`locacao_pagamentos`	Pagamento da locação no Stripe; `reserva_id` → `reservas`.
`premium_assinaturas`	**Liga usuário, plano e assinatura:** `usuario_id` (PK e FK → `usuarios`), `stripe_customer_id`, `stripe_subscription_id`, `stripe_price_id`, `status` e `paid_period_end`.
`stripe_eventos`, `stripe_locacao_eventos`	Id dos eventos já processados (idempotência); sem payload nem dado de cartão.
`reset_tokens`	Tokens de redefinição de senha; `usuario_id` → `usuarios`.

3.6 Segurança

Toda conta nasce LOCATARIO; o locatário pode habilitar o modo anunciante e virar PROPRIETARIO, e só um ADMIN muda papéis no painel. A tabela mostra o que a API permite a cada papel.

Ação	visitante	locatário	proprietário	admin
ver landing, catálogo e anúncios ativos	sim	sim	sim	sim
pedir reserva e pagar a locação aprovada	não	sim	sim (não do próprio anúncio)	não
cadastrar equipamento, anúncio e fotos	não	não (403)	sim	não
aprovar ou recusar reservas recebidas	não	não	sim (só as suas)	não
assinar o Pro e abrir o portal do cliente	não	sim	sim	sim
ver reserva ou anúncio privado de outro usuário	não	não (404)	não (404)	sim (painel)
moderar anúncios, mudar papéis, ver auditoria e MRR	não	não	não	sim

Isolamento por usuário. O id do usuário vem sempre de `req.usuario`, preenchido pelo auth-service, e nunca do corpo da requisição. Toda consulta de recurso privado filtra pelo dono na própria cláusula SQL, por exemplo `WHERE r.id = ? AND (r.solicitante_id = ? OR e.proprietario_id = ?)`. Quando o recurso não existe ou é de outra pessoa, a resposta é a mesma, 404 “Recurso não encontrado”, para não revelar que o id existe. Alterações usam `SELECT … FOR UPDATE` na mesma transação, e tentativas negadas por papel geram 403 e o evento `ACAO_NEGADA`.

Segredos. Chaves do Stripe, `JWT_SECRET`, senhas do banco, do MinIO e do SMTP ficam só nas variáveis da stack no Portainer. O repositório tem apenas `.env.example` com os nomes, o `.gitignore` exclui os arquivos `.env`, e o Stripe está em modo de teste: o código recusa objetos com `livemode: true`.

3.7 Observabilidade, CI/CD e documentação da API

Fiz as três atividades extras no primeiro bimestre e elas continuam ativas no GiroRural:

- **Health checks:** `/live` (processo vivo) e `/health` (prontidão, com 503 quando uma dependência falha) na API, no auth-service e no log-service. O `/health` da API testa MariaDB, auth-service, log-service e MinIO, e o Docker usa esses endpoints nos `healthcheck` da stack.
- **Métricas:** `/metrics` no formato Prometheus, com contador de requisições e histograma de latência por método, rota e status.
- **Swagger:** `/apidocs` (API do GiroRural) e `/apidocs/auth`, gerados a partir de anotações `@openapi` no código e validados no CI.
- **CI/CD:** o workflow `cicd.yml` roda em push e pull request: `npm ci` nos três serviços, `npm test`, `npm run docs:validate`, build das quatro imagens e, em push, publicação no GHCR com a tag `sha-<commit>`. O deploy é semiautomático: eu troco `IMAGE_TAG` na stack e clico em **Update the stack**.

3.8 Camadas IaaS, PaaS e SaaS do projeto

Camada	Onde aparece no GiroRural	Quem gerencia
IaaS	A VM da disciplina onde o Docker roda, com rede, disco e o subdomínio `*.lapps.studio`; o servidor MariaDB externo.	Professor / infraestrutura da Fatec
PaaS	Docker + Portainer como plataforma de execução: eu entrego um Compose e imagens, e a plataforma cuida de rede, volumes, reinício e health checks. O GHCR e o GitHub Actions fazem o papel de registro e pipeline gerenciados.	Professor (Portainer); GitHub (Actions e GHCR); eu configuro a stack
SaaS	O próprio GiroRural: o produtor usa pelo navegador, sem instalar nada, e paga uma assinatura mensal pelo plano Pro. O GiroRural também consome outros SaaS: Stripe (cobrança) e o provedor SMTP.	Eu (GiroRural); Stripe; provedor de e-mail

No recorte didático, a VM é IaaS e o Portainer faz o papel de PaaS, mas são componentes autogerenciados pela turma, não serviços contratados de um provedor de nuvem. Redis e MinIO também são autogerenciados, dentro da stack.

4 Quadro de entregas

Tabela gerada a partir das fichas da seção 5. Horário de Brasília (UTC−03).

Nº	Atividade	Data planejada	Data realizada	Situação
7	Plano Premium — cobrança de verdade com Stripe	09/10/2026	09/10/2026 20:29	entregue
8	SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve	16/10/2026	16/10/2026 17:39	entregue
9	Fork e plano de negócio	23/10/2026	23/10/2026 20:44	entregue
10	Seu negócio no ar — o domínio no lugar dos filmes	06/11/2026	06/11/2026 20:24	entregue

Nº	Atividade	Data planejada	Data realizada	Situação
11	SaaS completo — planos, limites e assinatura com Stripe	13/11/2026	12/11/2026 23:14	entregue

5 Atividades realizadas

5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe

Descrição	Plano Premium de R$ 9,90/mês no catálogo de filmes, cobrado pelo Stripe Checkout em modo de teste e ativado por webhook.
Data planejada	09/10/2026
Data realizada	09/10/2026 20:29
Situação	entregue
Evidência	Commits 1c2912a e ec5df7e (09/10/2026); checkout concluído no Stripe em 09/10/2026 22:38 com o cartão 4242; issue #6 do repositório catalogo-filmes; prints do checkout e do perfil Premium.
Link	https://github.com/Luanaabrantes/girorural/commit/ec5df7e596a380e2a16c5701e4db9fe4a98d501f

Links da entrega. Commit da entrega — ec5df7e (09/10/2026 20:29) · Commit da primeira versão — 1c2912a (09/10/2026 17:13) · Issue de entrega #6 (catalogo-filmes)

O que foi feito.

O catálogo de filmes ganhou um plano Premium mensal. O botão de assinar chama a API, que cria uma sessão do Stripe Checkout em modo de teste ligada ao usuário logado. O Premium não é ativado pelo retorno do navegador: o webhook validado com a assinatura do Stripe grava a assinatura em `premium_assinaturas` e registra o id do evento em `stripe_eventos`, para que o mesmo evento não seja processado duas vezes. Na segunda versão (ec5df7e) passei a guardar o período pago, as datas de renovação e cancelamento e um script de sincronização com o Stripe, e o perfil mostra o selo Premium com a validade.

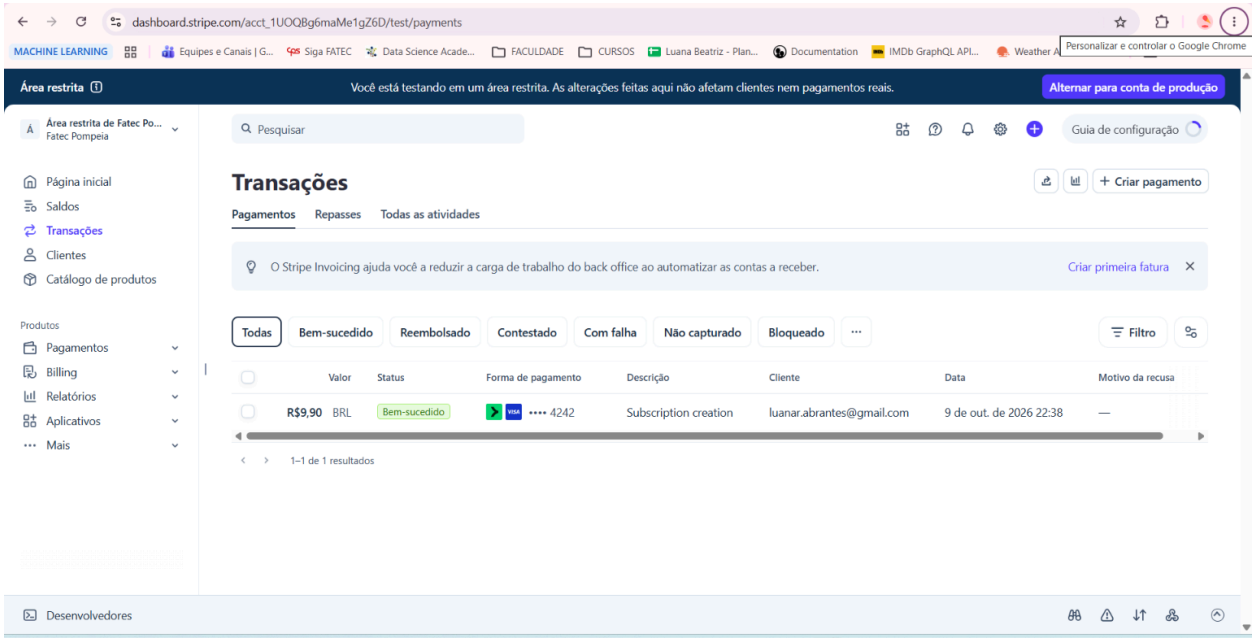


Figura 5: Atividade 7 — Transação de R$ 9,90 com o cartão 4242 concluída no Stripe (modo de teste) em 9 de outubro de 2026 às 22:38, criação da assinatura.

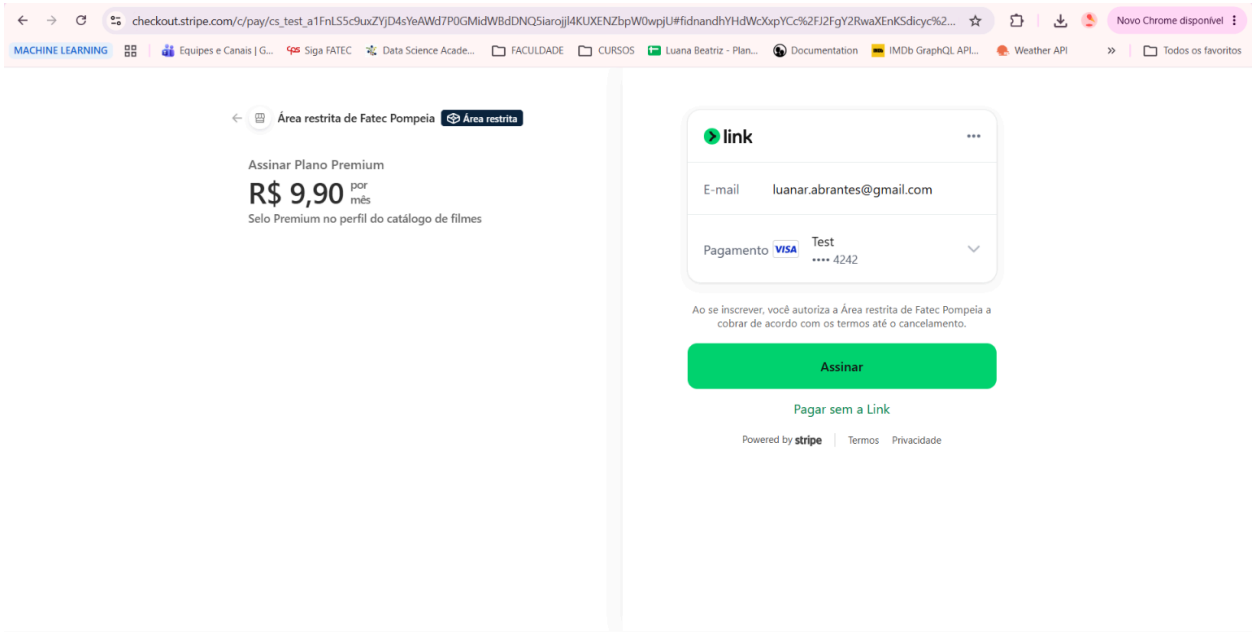


Figura 6: Atividade 7 — Stripe Checkout em modo de teste com o cartão 4242, assinatura do Plano Premium de R$ 9,90/mês.

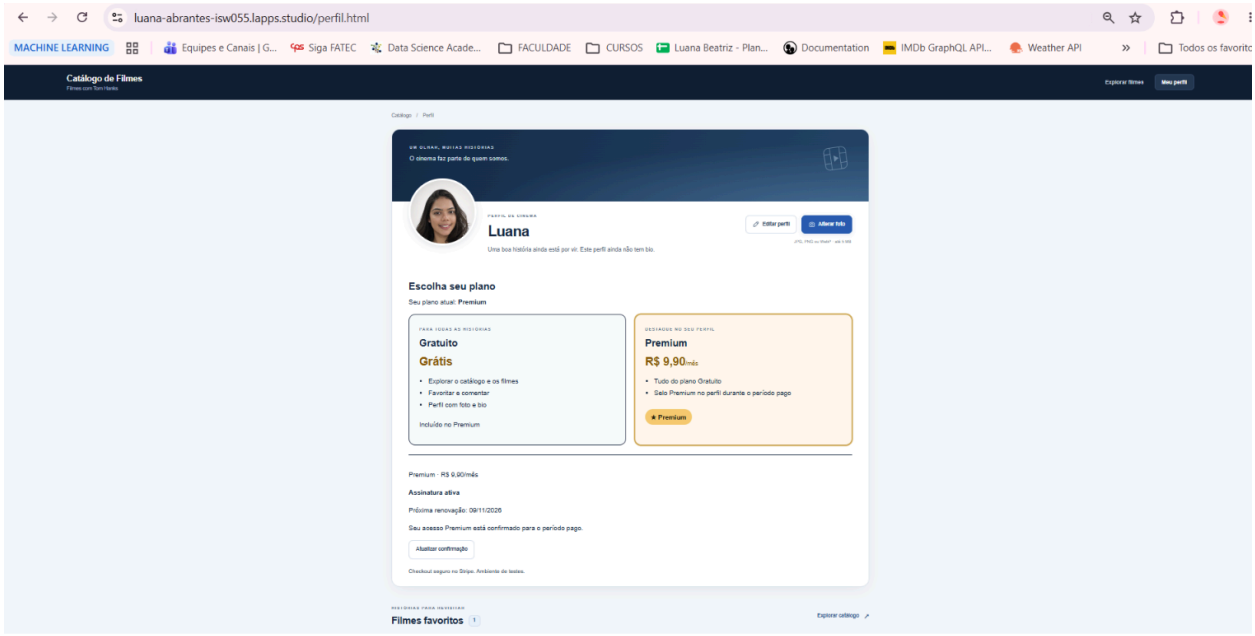


Figura 7: Atividade 7 — Perfil do usuário com o plano Premium ativo depois do webhook.

Dificuldades e como foram resolvidas.

A parte mais difícil foi entender que o retorno do checkout não prova pagamento: só o webhook assinado pode mudar o plano. Também precisei tratar eventos repetidos e fora de ordem, o que resolvi com a tabela de eventos e a marca de tempo do último evento aplicado. O código foi publicado e o checkout concluído no dia 09/10.

5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve

Descrição	Documentação reversa do catálogo de filmes (DOCUMENTACAO.md) e classificação SaaS/PaaS/IaaS, linkadas no README.
Data planejada	16/10/2026
Data realizada	16/10/2026 17:39
Situação	entregue
Evidência	Commit 879e25e com o DOCUMENTACAO.md e o link no README; merge na main em 16/10/2026 20:10 (9c024a0).
Link	https://github.com/Luanaabrantes/girorural/commit/879e25e2d087e3af0ca932365708b29b31e17341

Links da entrega. Commit da entrega — 879e25e · DOCUMENTACAO.md

O que foi feito.

Escrevi a documentação que o catálogo de filmes não tinha, lendo o próprio código: escopo e histórico, rotas, diagrama de arquitetura em Mermaid, regras de negócio de autenticação, catálogo, auditoria, Premium e observabilidade, modelo de dados, fluxos principais e limitações conhecidas. No README entrou o link para o `DOCUMENTACAO.md` e a tabela de classificação: o catálogo como SaaS, a VM como IaaS e Docker com Portainer como PaaS de execução, com a ressalva de que os componentes são autogerenciados.

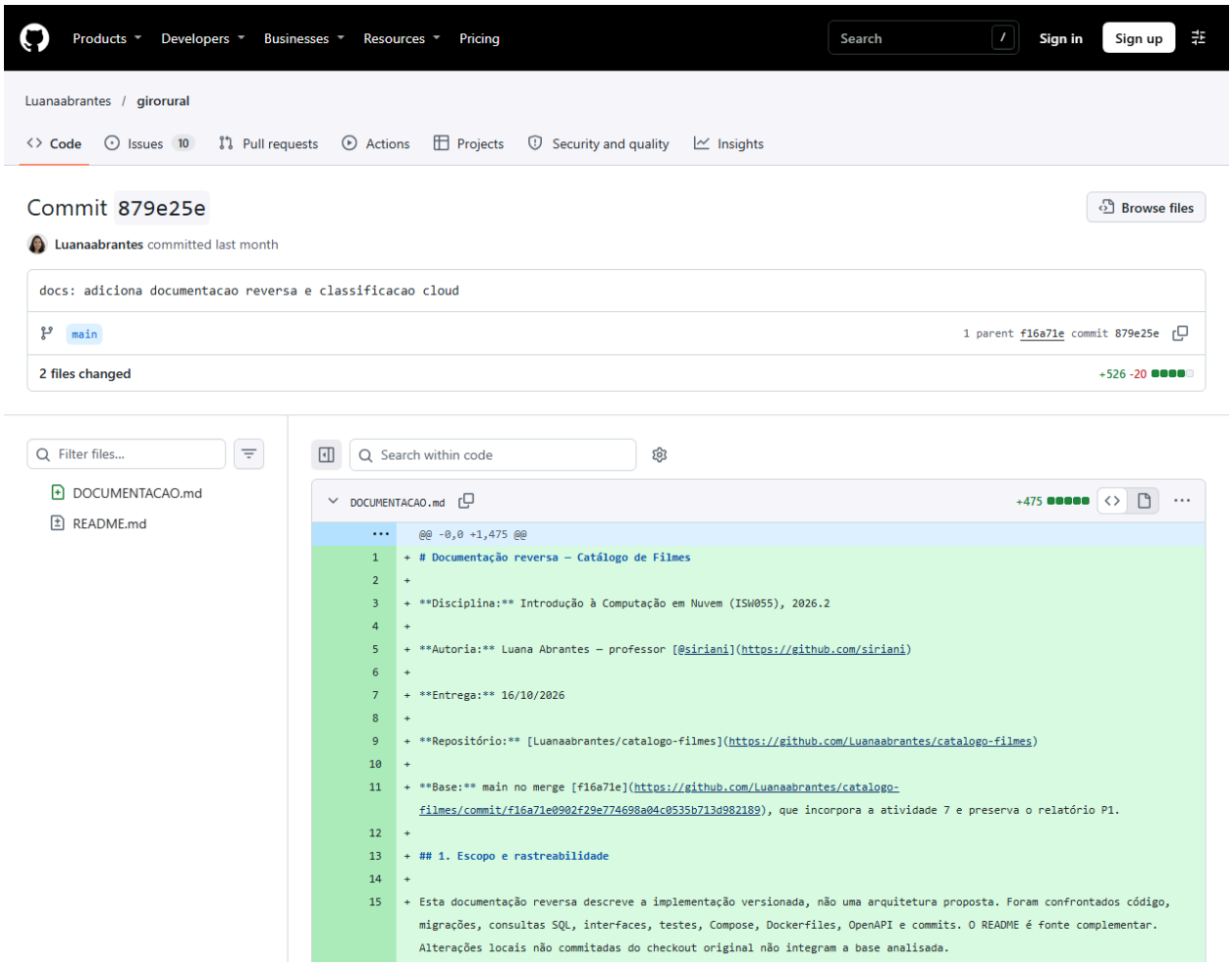


Figura 8: Atividade 8 — Commit 879e25e no repositório Luanaabrantes/girorural: página do GitHub e, abaixo, a API com “date”: “2026-10-16T20:39:01Z” (16/10/2026 17:39 em Brasília) (1/2: página do GitHub).

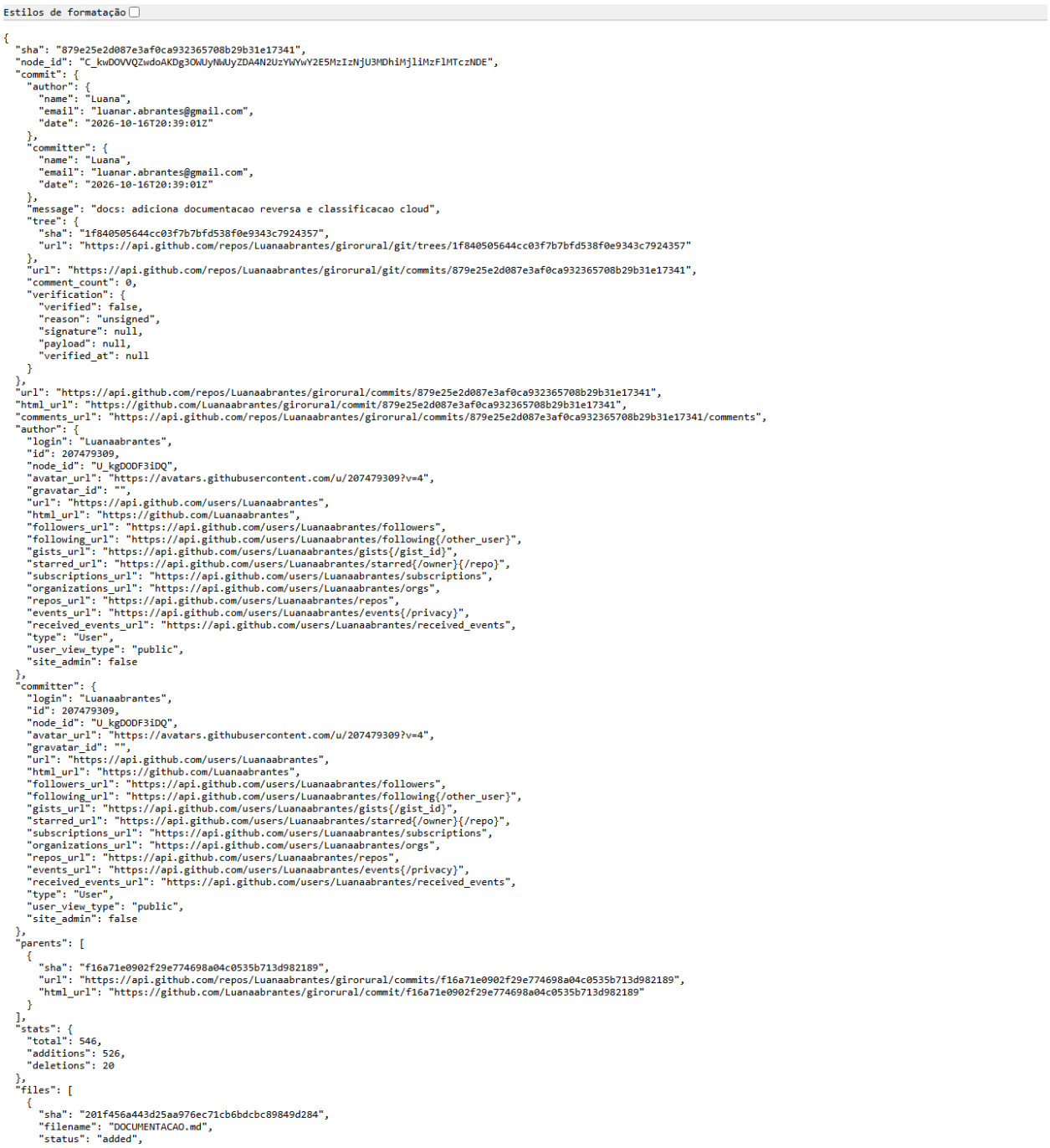


Figura 9: Atividade 8 — Commit 879e25e no repositório Luanaabrantes/girorural: página do GitHub e, abaixo, a API com “date”: “2026-10-16T20:39:01Z” (16/10/2026 17:39 em Brasília) (2/2: resposta da API do GitHub com a data e a hora em UTC).

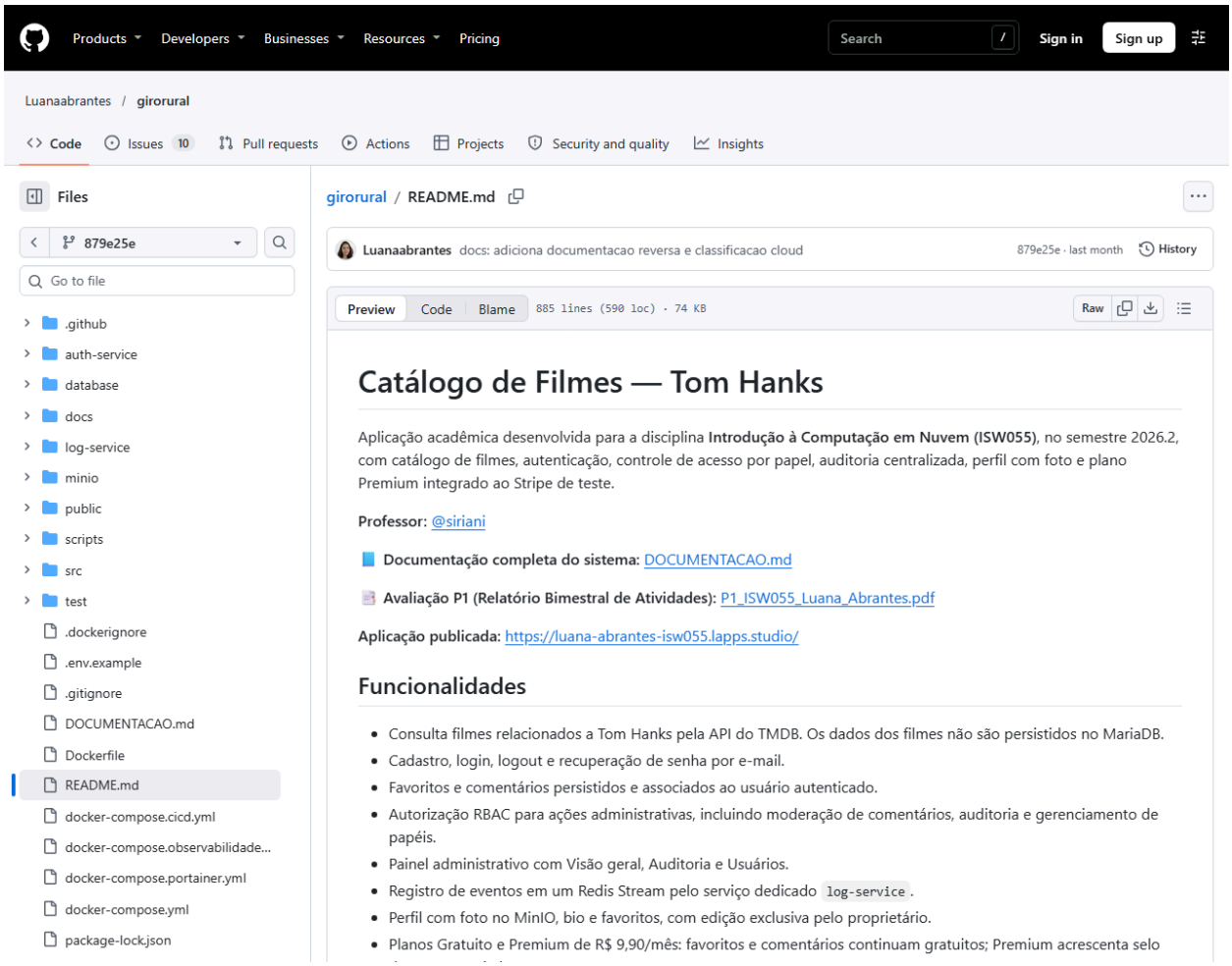


Figura 10: Atividade 8 — README no commit 879e25e com o link "Documentação completa do sistema: DOCUMENTACAO.md".

Dificuldades e como foram resolvidas.

A maior dificuldade foi a leitura reversa do código. O catálogo tinha crescido uma atividade por vez, em branches diferentes, e precisei voltar ao histórico de commits para entender em que ordem cada parte entrou e o que cada serviço fazia de fato. Ao conferir os arquivos, percebi que algumas coisas que eu dava como certas não batiam com o código: o MariaDB, por exemplo, não está no Docker Compose, é um banco externo à stack. Também tive dúvida na classificação das camadas, porque o mesmo componente muda de camada conforme quem o gerencia. Defini a VM da disciplina como IaaS, o Docker com o Portainer como PaaS e o catálogo como SaaS para quem usa. Para resolver, conferi cada afirmação nos arquivos do repositório e registrei no próprio documento as limitações e o que não dava para comprovar.

5.3 Atividade 9 — Fork e plano de negócio

Descrição	Repositório novo com o histórico completo do catálogo, README do produto GiroRural, plano de implementação e issues de desenvolvimento.
Data planejada	23/10/2026
Data realizada	23/10/2026 20:44
Situação	entregue
Evidência	Issue #10 "Entrega — Fork e plano de negócio" mencionando @siriani; commit do README 7b27eed (23/10/2026 19:18); issues #1 a #7.

Link	https://github.com/Luanaabrantes/girorural/issues/10

Links da entrega. Issue de entrega 10 · Commit do README e planejamento — 7b27eed · Issues de desenvolvimento

O que foi feito.

Criei o repositório `Luanaabrantes/girorural` com a cópia integral do histórico do catálogo de filmes, desde o commit 0e3b62a de 20/08/2026, e escrevi o README apresentando o GiroRural, o plano de implementação e a inspeção da arquitetura herdada. Abri sete issues de desenvolvimento (modelo de dados e isolamento, cadastro de equipamentos e anúncios, fotos, busca e filtros, reservas, painel e assinatura com Stripe). O plano previa inicialmente um plano Free com dois anúncios ativos e um Pro com até vinte; na atividade 11 esse modelo foi ajustado para três anúncios novos por mês no Gratuito e Pro sem limite.

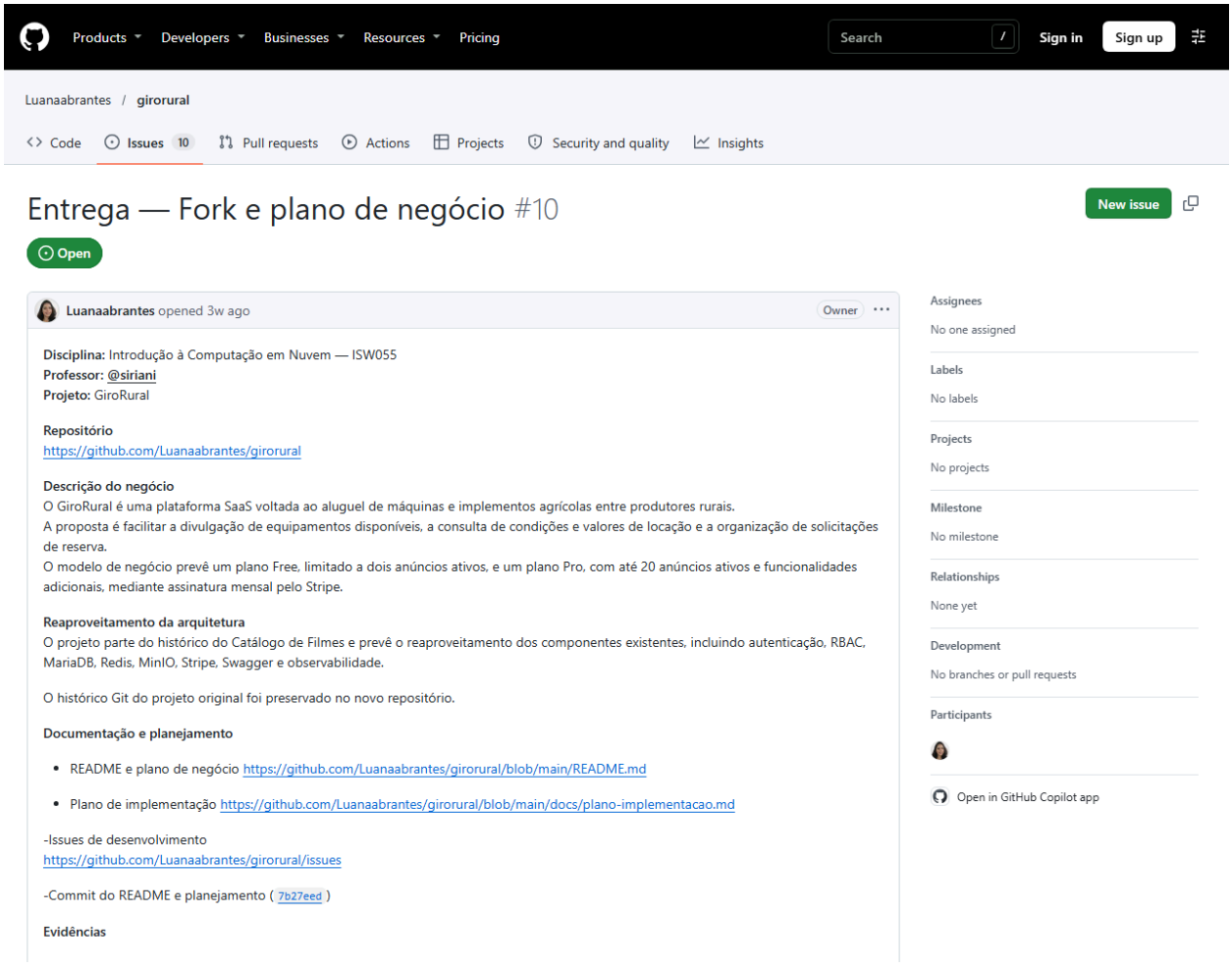


Figura 11: Atividade 9 — Issue de entrega 10 mencionando @siriani: página do GitHub e, abaixo, a API com "created_at": "2026-10-23T23:44:42Z" (23/10/2026 20:44 em Brasília) (1/2: página do GitHub).

Estilos de formatação

```
{
  "url": "https://api.github.com/repos/Luanaabrantes/girorural/issues/10",
  "repository_url": "https://api.github.com/repos/Luanaabrantes/girorural",
  "labels_url": "https://api.github.com/repos/Luanaabrantes/girorural/issues/10/labels{/name}",
  "comments_url": "https://api.github.com/repos/Luanaabrantes/girorural/issues/10/comments",
  "events_url": "https://api.github.com/repos/Luanaabrantes/girorural/issues/10/events",
  "html_url": "https://github.com/Luanaabrantes/girorural/issues/10",
  "id": 6007108992,
  "node_id": "I_kwDOVVQZwc8AAAABZg01gA",
  "number": 10,
  "title": "Entrega — Fork e plano de negócio",
  "user": {
    "login": "Luanaabrantes",
    "id": 207479309,
    "node_id": "U_kgDODF3iDQ",
    "avatar_url": "https://avatars.githubusercontent.com/u/207479309?v=4",
    "gravatar_id": "",
    "url": "https://api.github.com/users/Luanaabrantes",
    "html_url": "https://github.com/Luanaabrantes",
    "followers_url": "https://api.github.com/users/Luanaabrantes/followers",
    "following_url": "https://api.github.com/users/Luanaabrantes/following{/other_user}",
    "gists_url": "https://api.github.com/users/Luanaabrantes/gists{/gist_id}",
    "starred_url": "https://api.github.com/users/Luanaabrantes/starred{/owner}{/repo}",
    "subscriptions_url": "https://api.github.com/users/Luanaabrantes/subscriptions",
    "organizations_url": "https://api.github.com/users/Luanaabrantes/orgs",
    "repos_url": "https://api.github.com/users/Luanaabrantes/repos",
    "events_url": "https://api.github.com/users/Luanaabrantes/events{/privacy}",
    "received_events_url": "https://api.github.com/users/Luanaabrantes/received_events",
    "type": "User",
    "user_view_type": "public",
    "site_admin": false
  },
  "labels": [

  ],
  "state": "open",
  "locked": false,
  "assignees": [

  ],
  "milestone": null,
  "comments": 0,
  "created_at": "2026-10-23T23:44:42Z",
  "updated_at": "2026-10-23T23:44:42Z",
  "closed_at": null,
  "assignee": null,
  "author_association": "OWNER",
  "active_lock_reason": null,
  "sub_issues_summary": {
    "total": 0,
    "completed": 0,
    "percent_completed": 0
  },
  "issue_dependencies_summary": {
    "blocked_by": 0,
    "total_blocked_by": 0,
    "blocking": 0,
    "total_blocking": 0
  },
  "body": "**Disciplina:** Introdução à Computação em Nuvem — ISW055\n**Professor:** @siriani\n**Projeto:**
GiroRural\n\n**Repositório**\nhttps://github.com/Luanaabrantes/girorural\n\n**Descrição do negócio**\nO GiroRural é uma plataforma SaaS voltada ao aluguel de máquinas e
implementos agrícolas entre produtores rurais.\nA proposta é facilitar a divulgação de equipamentos disponíveis, a consulta de condições e valores de locação e a organização de
solicitações de reserva.\nO modelo de negócio prevê um plano Free, limitado a dois anúncios ativos, e um plano Pro, com até 20 anúncios ativos e funcionalidades adicionais,
mediante assinatura mensal pelo Stripe.\n\n**Reaproveitamento da arquitetura**\nO projeto parte do histórico do Catálogo de Filmes e prevê o reaproveitamento dos componentes
existentes, incluindo autenticação, RBAC, MariaDB, Redis, MinIO, Stripe, Swagger e observabilidade.\n\nO histórico Git do projeto original foi preservado no novo
repositório.\n\n**Documentação e planejamento**\n- README e plano de negócio https://github.com/Luanaabrantes/girorural/blob/main/README.md\n\n- Plano de implementação
https://github.com/Luanaabrantes/girorural/blob/main/docs/plano-implementacao.md\n\n-Issues de desenvolvimento \nhttps://github.com/Luanaabrantes/girorural/issues\n\n-Commit do
README e planejamento (https://github.com/Luanaabrantes/girorural/commit/7b27eedcb2c626ddf3a2db7c5351bed161ae943e)\n\n**Evidências**\n\nEvidência 1 — Histórico de commits\n!
[Histórico de commits](../blob/main/docs/evidencias/atividade-9-commits.png)\n\nEvidência 2 — Issues de desenvolvimento\n![Issues de desenvolvimento]
(../blob/main/docs/evidencias/atividade-9-issues-desenvolvimento.png)",
  "closed_by": null,
  "reactions": {
    "url": "https://api.github.com/repos/Luanaabrantes/girorural/issues/10/reactions",
    "total_count": 0,
    "+1": 0,
    "-1": 0,
    "laugh": 0,
    "hooray": 0,
    "confused": 0,
    "heart": 0,
    "rocket": 0,
    "eyes": 0
  },
  "timeline_url": "https://api.github.com/repos/Luanaabrantes/girorural/issues/10/timeline",
  "performed_via_github_app": null,
  "state_reason": null,
  "pinned_comment": null
}
```

Figura 12: Atividade 9 — Issue de entrega 10 mencionando @siriani: página do GitHub e, abaixo, a API com “created_at”: “2026-10-23T23:44:42Z” (23/10/2026 20:44 em Brasília) (2/2: resposta da API do GitHub com a data e a hora em UTC).

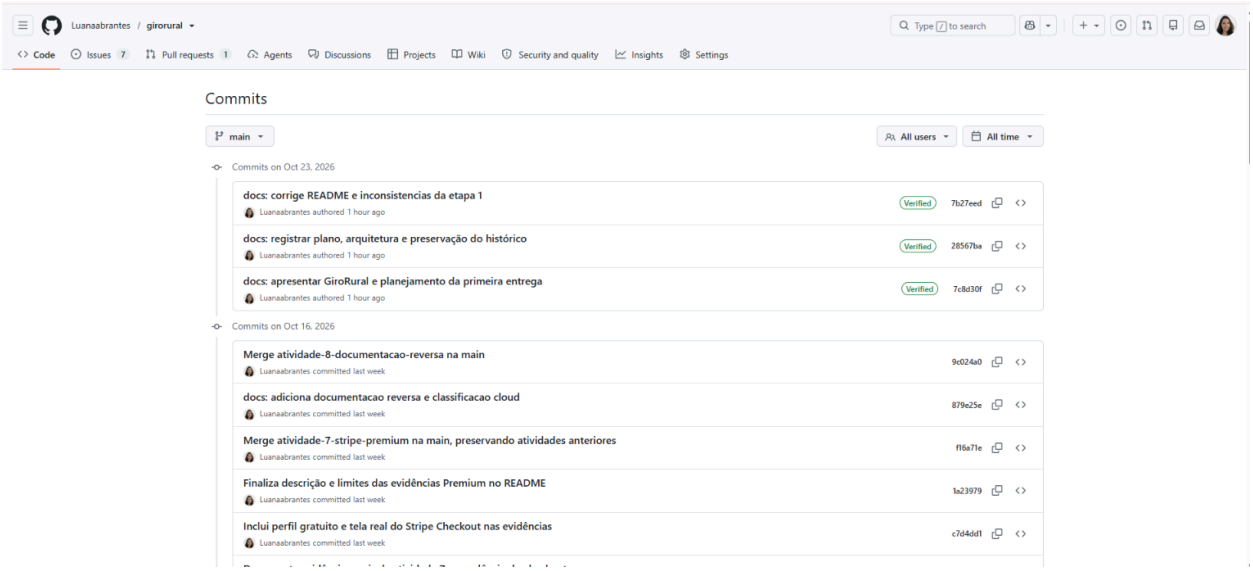


Figura 13: Atividade 9 — Histórico de commits preservado no repositório novo.

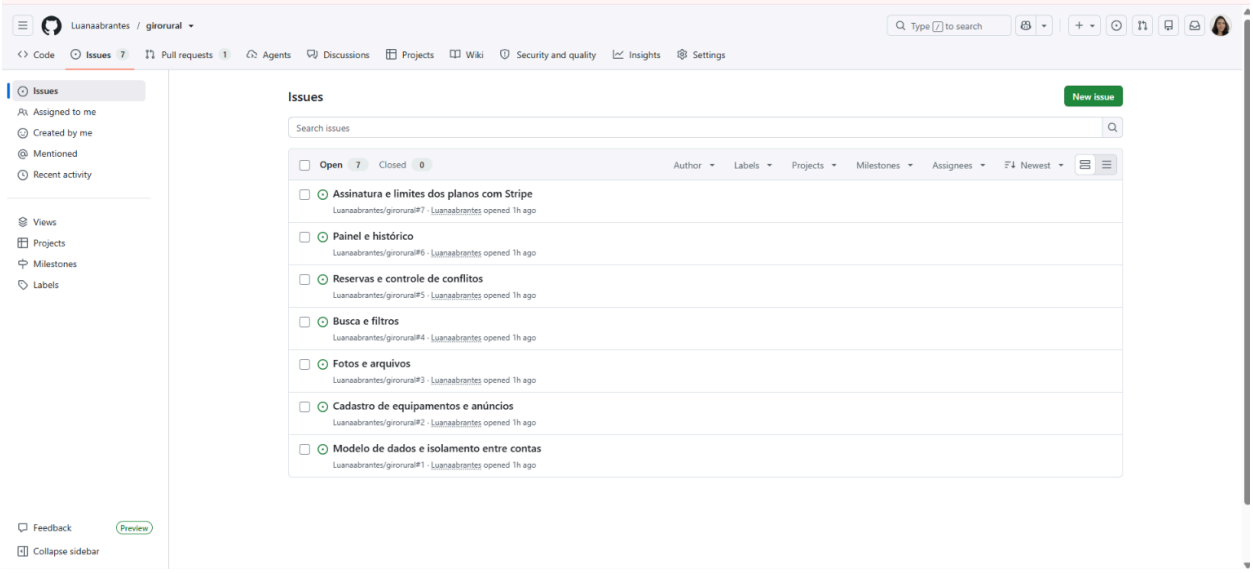


Figura 14: Atividade 9 — Issues de desenvolvimento abertas no repositório do produto.

Dificuldades e como foram resolvidas.

O desafio foi copiar o histórico sem perder branches e datas, e conferir que os hashes eram os mesmos nos dois repositórios. Um erro meu ficou na issue: as imagens dela apontam para `docs/evidencias` na `main`, mas o commit dos prints (7ab1fd6, 23/10 20:33) ficou na branch `docs/etapa-1-girorural`, então elas não abrem pela issue. Os prints existem nessa branch e estão reproduzidos acima.

5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes

Descrição	GiroRural publicado na mesma stack e subdomínio: anúncios, reservas, fotos no MinIO, papéis, 403 para quem não pode e eventos de negócio no log-service.
Data planejada	06/11/2026
Data realizada	06/11/2026 20:24
Situação	entregue

Evidência	Issue #11 "Entrega — Seu negócio no ar" mencionando @siriani; commits cb2ecfd (ACAO_NEGADA) e fa19e76 (galeria de evidências), 06/11/2026; URL no ar.
Link	https://github.com/Luanaabrantes/girorural/issues/11

Links da entrega. Issue de entrega 11 · Commit — fa19e76 (06/11/2026 20:27) · Sistema no ar · Galeria de evidências

O que foi feito.

O domínio do GiroRural foi implementado sobre a arquitetura herdada: equipamentos, anúncios com galeria de fotos no MinIO, busca com filtros, reservas com controle de conflito de datas, aprovação pelo proprietário, pagamento da locação (simulado na demonstração) e liberação do contato do dono só depois do pagamento. Os papéis passaram a ser LOCATARIO, PROPRIETARIO e ADMIN; toda conta nasce locatária e pode habilitar o modo anunciante. Tentativas de operação sem papel adequado recebem 403 e geram `ACAO_NEGADA` no log-service, e cada ação de negócio gera um evento de auditoria. A stack ficou com limite de 1 GB por contêiner e um catálogo inicial de 33 máquinas.

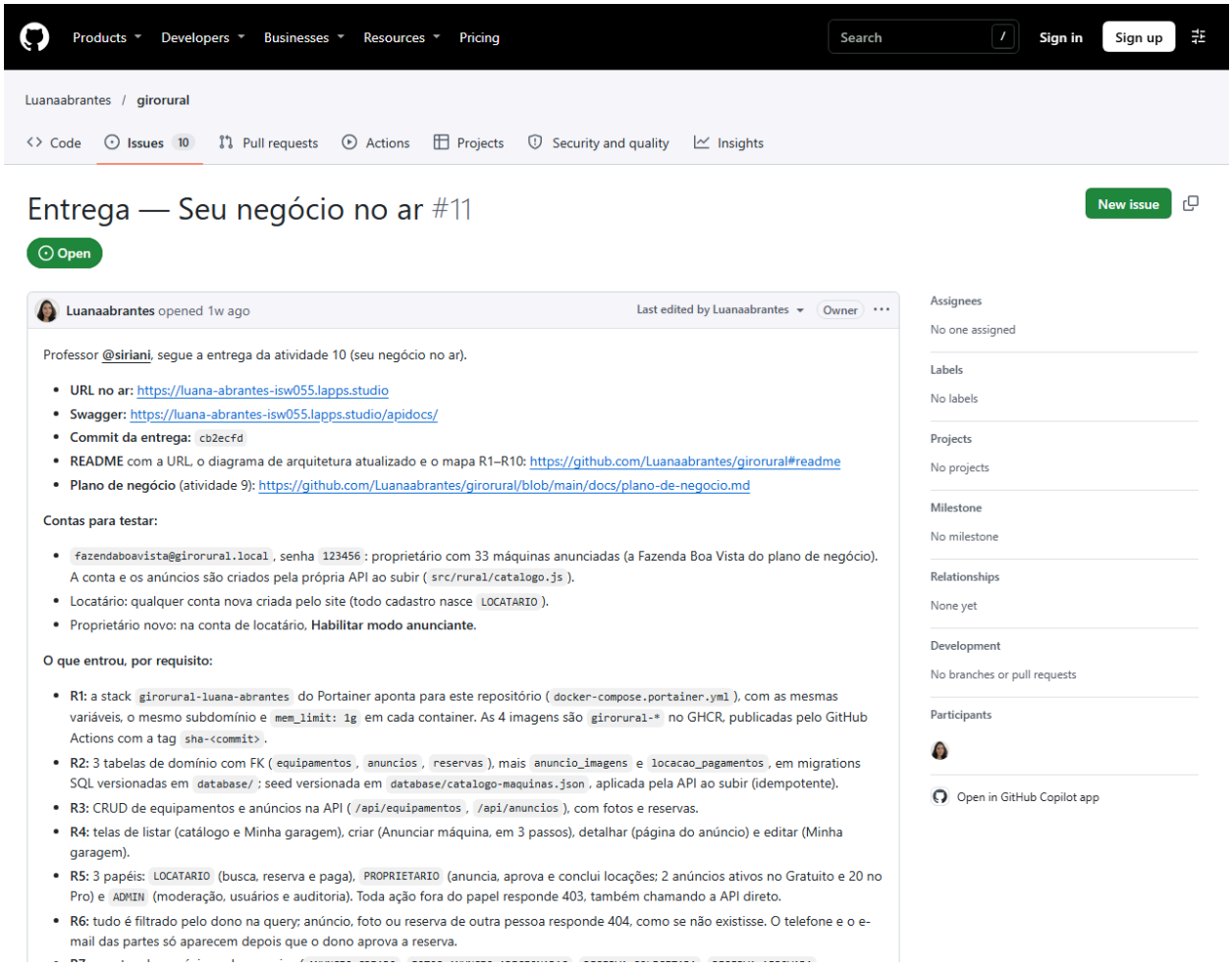


Figura 15: Atividade 10 — Issue de entrega 11 mencionando @siriani: página do GitHub e, abaixo, a API com "created_at": "2026-11-06T23:24:18Z" (06/11/2026 20:24 em Brasília) (1/2: página do GitHub).

Estilos de formatação


```
{
  "url": "https://api.github.com/repos/Luanaabrantes/girorural/issues/11",
  "repository_url": "https://api.github.com/repos/Luanaabrantes/girorural",
  "labels_url": "https://api.github.com/repos/Luanaabrantes/girorural/issues/11/labels{/name}",
  "comments_url": "https://api.github.com/repos/Luanaabrantes/girorural/issues/11/comments",
  "events_url": "https://api.github.com/repos/Luanaabrantes/girorural/issues/11/events",
  "html_url": "https://github.com/Luanaabrantes/girorural/issues/11",
  "id": 6252451387,
  "node_id": "I_kwDOVVQZwc8AAAABdKzWOw",
  "number": 11,
  "title": "Entrega — Seu negócio no ar",
  "user": {
    "login": "Luanaabrantes",
    "id": 207479309,
    "node_id": "U_kgDODF3iDQ",
    "avatar_url": "https://avatars.githubusercontent.com/u/207479309?v=4",
    "gravatar_id": "",
    "url": "https://api.github.com/users/Luanaabrantes",
    "html_url": "https://github.com/Luanaabrantes",
    "followers_url": "https://api.github.com/users/Luanaabrantes/followers",
    "following_url": "https://api.github.com/users/Luanaabrantes/following{/other_user}",
    "gists_url": "https://api.github.com/users/Luanaabrantes/gists{/gist_id}",
    "starred_url": "https://api.github.com/users/Luanaabrantes/starred{/owner}{/repo}",
    "subscriptions_url": "https://api.github.com/users/Luanaabrantes/subscriptions",
    "organizations_url": "https://api.github.com/users/Luanaabrantes/orgs",
    "repos_url": "https://api.github.com/users/Luanaabrantes/repos",
    "events_url": "https://api.github.com/users/Luanaabrantes/events{/privacy}",
    "received_events_url": "https://api.github.com/users/Luanaabrantes/received_events",
    "type": "User",
    "user_view_type": "public",
    "site_admin": false
  },
  "labels": [

  ],
  "state": "open",
  "locked": false,
  "assignees": [

  ],
  "milestone": null,
  "comments": 0,
  "created_at": "2026-11-06T23:24:18Z",
  "updated_at": "2026-11-06T23:27:30Z",
  "closed_at": null,
  "assignee": null,
  "author_association": "OWNER",
  "active_lock_reason": null,
  "sub_issues_summary": {
    "total": 0,
    "completed": 0,
    "percent_completed": 0
  },
  "issue_dependencies_summary": {
    "blocked_by": 0,
    "total_blocked_by": 0,
    "blocking": 0,
    "total_blocking": 0
  },
  "body": "Professor @siriani, segue a entrega da atividade 10 (seu negócio no ar).\n\n- **URL no ar:** https://luana-abrantes-isw055.lapps.studio\n- **Swagger:** https://luana-abrantes-isw055.lapps.studio/apidocs/\n- **Commit da entrega:** `cb2ecfd`\n- **README** com a URL, o diagrama de arquitetura atualizado e o mapa R1–R10: https://github.com/Luanaabrantes/girorural#readme\n- **Plano de negócio** (atividade 9): https://github.com/Luanaabrantes/girorural/blob/main/docs/plano-de-negocio.md\n\n**Contas para testar:**\n\n- `fazendaboavista@girorural.local`, senha `123456`: proprietário com 33 máquinas anunciadas (a Fazenda Boa Vista do plano de negócio). A conta e os anúncios são criados pela própria API ao subir (`src/rural/catalogo.js`).\n- Locatário: qualquer conta nova criada pelo site (todo cadastro nasce `LOCATARIO`).\n- Proprietário novo: na conta de locatário, **Habilitar modo anunciante**.\n\n**O que entrou, por requisito:**\n\n- **R1:** a stack `girorural-luana-abrantes` do Portainer aponta para este repositório (`docker-compose.portainer.yml`), com as mesmas variáveis, o mesmo subdomínio e `mem_limit: 1g` em cada container. As 4 imagens são `girorural-*` no GHCR, publicadas pelo GitHub Actions com a tag `sha-<commit>`.\n- **R2:** 3 tabelas de domínio com FK (`equipamentos`, `anuncios`, `reservas`), mais `anuncio_imagens` e `locacao_pagamentos`, em migrations SQL versionadas em `database/`; seed versionada em `database/catalogo-maquinas.json`, aplicada pela API ao subir (idempotente).\n- **R3:** CRUD de equipamentos e anúncios na API (`/api/equipamentos`, `/api/anuncios`), com fotos e reservas.\n- **R4:** telas de listar (catálogo e Minha garagem), criar (Anunciar máquina, em 3 passos), detalhar (página do anúncio) e editar (Minha garagem).\n- **R5:** 3 papéis: `LOCATARIO` (busca, reserva e paga), `PROPRIETARIO` (anuncia, aprova e conclui locações; 2 anúncios ativos no Gratuito e 20 no Pro) e `ADMIN` (moderação, usuários e auditoria). Toda ação fora do papel responde 403, também chamando a API direto.\n- **R6:** tudo é filtrado pelo dono na query; anúncio, foto ou reserva de outra pessoa responde 404, como se não existisse. O telefone e o e-mail das partes só aparecem depois que o dono aprova a reserva.\n- **R7:** eventos de negócio no log-service (`ANUNCIO_CRIADO`, `FOTOS_ANUNCIO_ADICIONADAS`, `RESERVA_SOLICITADA`, `RESERVA_APROVADA`, `LOCACAO_PAGA`, `RESERVA_CONCLUIDA`, `ACAO_NEGADA`…).\n- **R8:** fotos do anúncio no bucket privado do MinIO (até 12 por anúncio, regravadas no servidor e sem EXIF), ligadas ao anúncio pela tabela `anuncio_imagens`; o navegador só acessa pela API.\n- **R9:** landing pública com a proposta de valor, as categorias, as máquinas em destaque e o botão de cadastro.\n- **R10:** telas, rotas, serviços e textos do catálogo de filmes removidos; a variável da API externa saiu do compose e do Portainer. Só o nome da rede Docker `catalogo-network` foi mantido, para não recriar a rede da stack.\n\nO banco é o MySQL remoto da disciplina (no lugar do MariaDB), o mesmo das atividades anteriores. O CI roda 103 testes (api, auth-service e log-service) e valida o contrato OpenAPI antes de construir e publicar as imagens.\n\n**Pagamento:** o locatário paga a locação depois da aprovação do dono, pelo Stripe Checkout em modo de teste (cartão 4242 4242 4242 4242), com confirmação só pelo webhook assinado. A tela também oferece pagamento simulado (aprovado/recusado) para demonstração. O plano Pro do anunciante (R$ 39,90/mês) é uma assinatura Stripe.\n\n**Prints:** todos estão na pasta de evidências, cada um com a legenda abaixo: https://github.com/Luanaabrantes/girorural/tree/main/docs/evidencias/atividade-10\n\n- Landing pública com a proposta de valor e o botão de cadastro\n- Catálogo com as máquinas anunciadas e os filtros\n- Minha garagem da Fazenda Boa Vista, com as solicitações recebidas\n- Cadastro de máquina (Anunciar máquina, passo 1)\n- 403: o José (locatário) chama `POST /api/anuncios` direto pela API. A API responde 403 \"Papel sem permissão para esta operação\"\n- 404: o José abre a reserva #1, que é de outra pessoa. A tela e a API respondem 404 \"Reserva não encontrada\"\n- Upload ligado ao registro: foto enviada para o anúncio #34 (vira a foto principal)\n- Arquivo no object storage: a foto do anúncio #34 é lida pela API no bucket privado do MinIO (`GET /api/anuncios/34/fotos/1`, 200, image/jpeg); o navegador nunca acessa o MinIO direto\n- Fluxo de locação: o José pede a reserva, a Fazenda aprova e o contato das duas partes é liberado\n- Eventos de negócio no log-service (painel de admin, aba Auditoria): reserva solicitada, aprovada, equipamento e anúncio criados, fotos enviadas e o `ACAO_NEGADA` de uma ação sem permissão\n- Portainer: os 5 containers da stack `girorural-luana-abrantes` saudáveis, com as imagens `girorural-*:sha-cb2ecfd…`\n\n**Próximos passos:** avaliação entre locatário e proprietário depois da locação, e repasse automático ao proprietário com Stripe Connect.\n\n\n---\n\nGenerated by [Claude Code](https://claude.ai/code)_",
  "closed_by": null,
  "reactions": {
    "url": "https://api.github.com/repos/Luanaabrantes/girorural/issues/11/reactions",
```


Figura 16: Atividade 10 — Issue de entrega 11 mencionando @siriani: página do GitHub e, abaixo, a API com “created_at”: “2026-11-06T23:24:18Z” (06/11/2026 20:24 em Brasília) (2/2: resposta da API do GitHub com a data e a hora em UTC).



Figura 17: Atividade 10 — Landing do GiroRural no ar no subdomínio da disciplina.

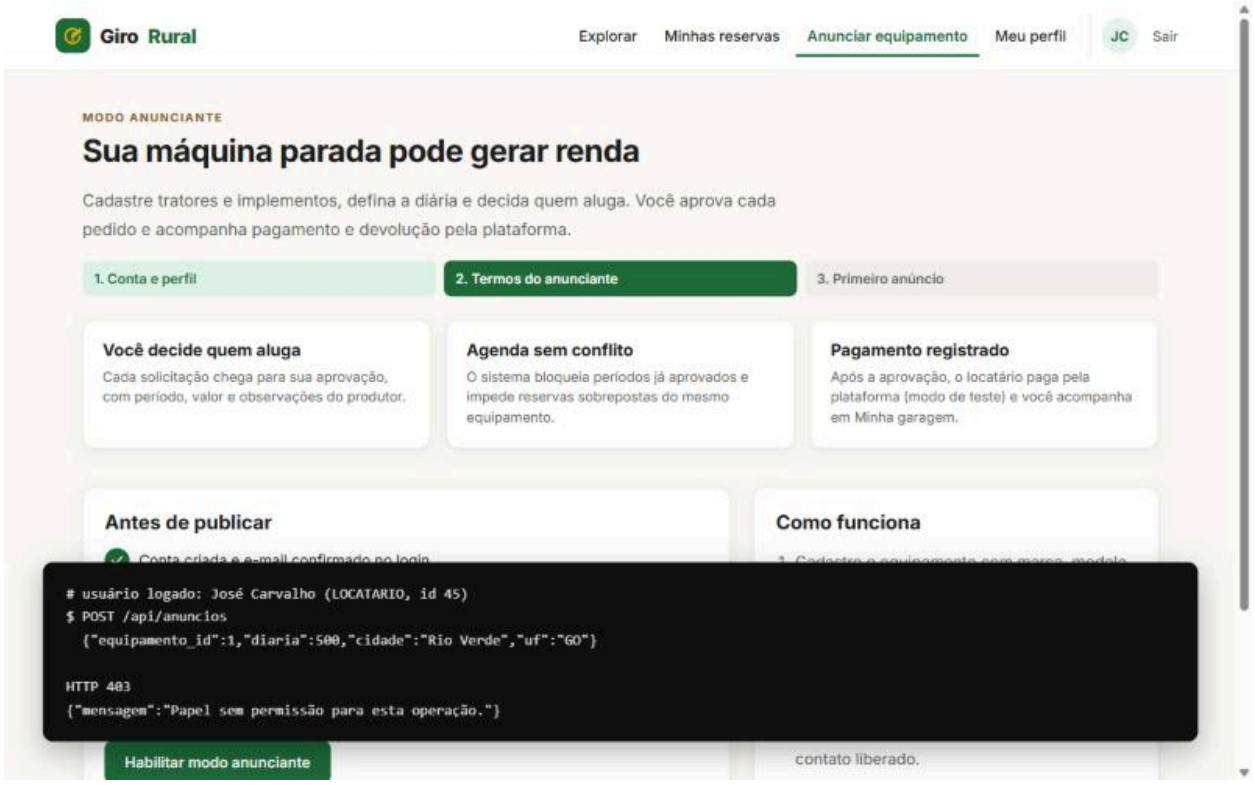


Figura 18: Atividade 10 — Locatário tentando anunciar: a API responde 403.

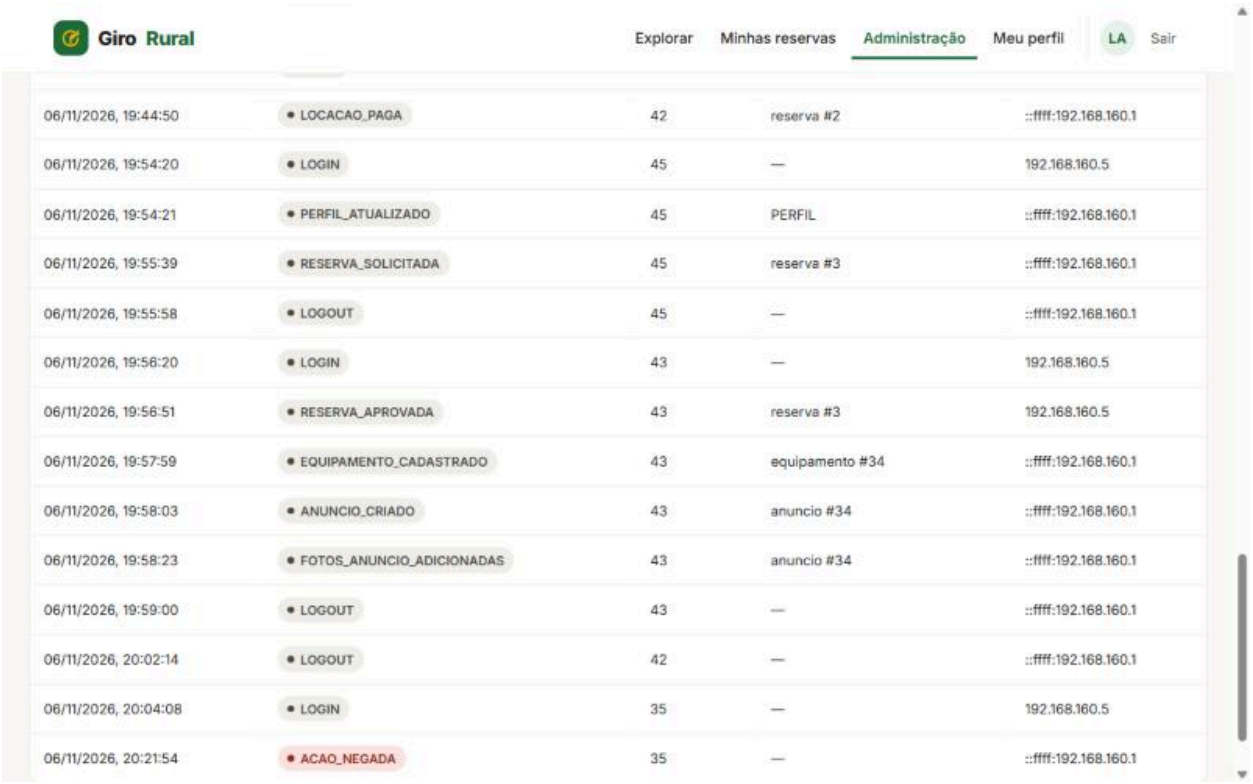


Figura 19: Atividade 10 — Eventos de negócio e ACAO_NEGADA consultados no log-service.

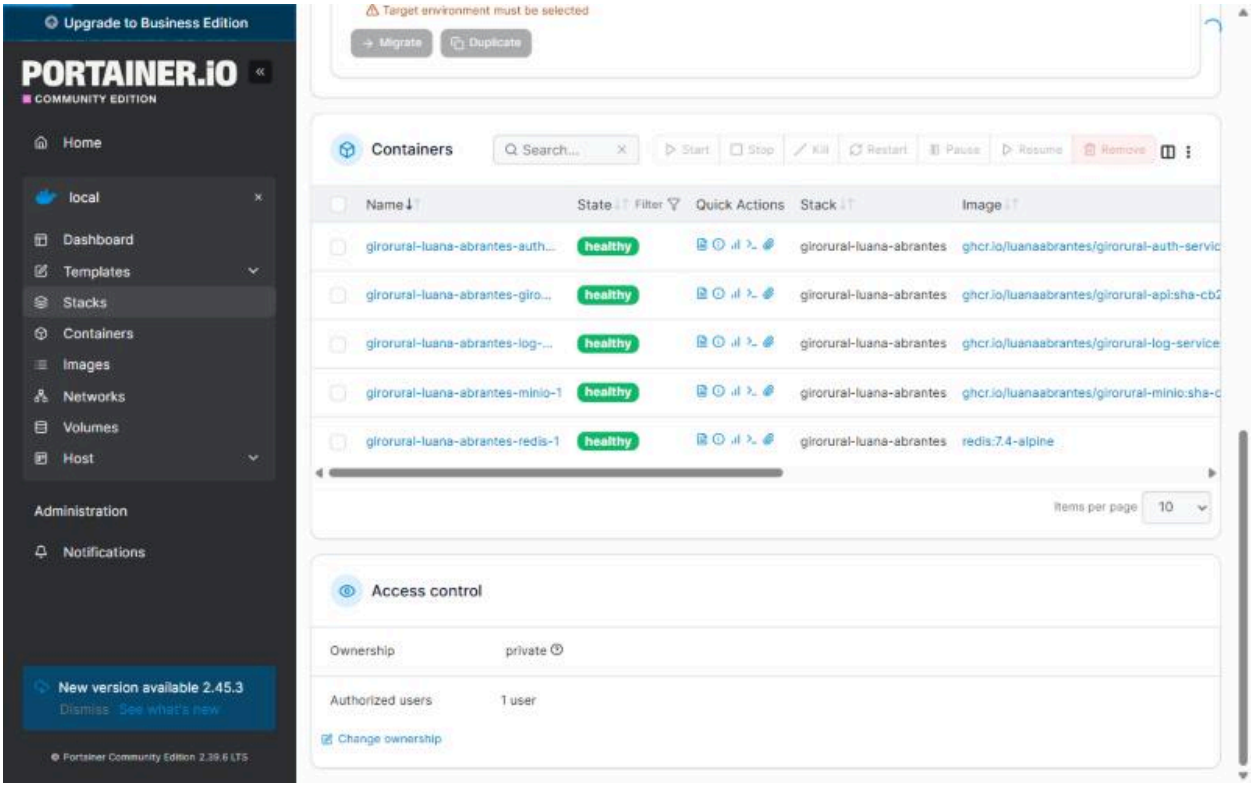


Figura 20: Atividade 10 — Contêineres da stack girorural-luana-abrantes no Portainer.

Dificuldades e como foram resolvidas.

As migrações precisavam funcionar tanto no MariaDB da disciplina quanto no MySQL 8 local, o que exigiu reescrever partes do SQL (commit ba0623c). A rede da stack colidiu com outras stacks por causa de uma subnet fixa, que removi (74a25d0). Também ajustei o workflow para publicar as imagens a partir da

branch da atividade e permitir disparo manual. Por fim, o 403 não aparecia no log porque o evento não era registrado na negação por papel; o commit cb2ecfd passou a registrar `ACAO_NEGADA`.

5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe

Descrição	Planos Gratuito (3 anúncios/mês) e Pro (R$ 39,90/mês, sem limite), limite aplicado na API, webhooks idempotentes, portal do cliente e painel de MRR.
Data planejada	13/11/2026
Data realizada	12/11/2026 23:14
Situação	entregue
Evidência	Issue #13 "Entrega — SaaS completo" mencionando @siriani; PR #12 incorporado à main no commit 0aac7bb (12/11/2026 21:31); prints da assinatura com o cartão 4242, dos webhooks e do limite.
Link	https://github.com/Luanaabrantes/girorural/issues/13

Links da entrega. Issue de entrega 13 · Commit — 0aac7bb (PR 12) · Prints da entrega (branch entrega--saas-prints)

O que foi feito.

O Premium da atividade 7 virou um modelo de receita completo. Os planos estão definidos num lugar só (`src/planos.js`). O checkout cria uma assinatura mensal recorrente, o webhook trata a criação, a atualização, o cancelamento, a falha de pagamento e a renovação, e o limite do plano é conferido na API a cada criação de anúncio, devolvendo 402 com uma mensagem que explica como liberar. O botão **Gerenciar assinatura** abre o portal do cliente do Stripe para trocar de plano, atualizar o cartão ou cancelar, e o painel administrativo ganhou a aba **Assinaturas e MRR** com assinantes, status e os últimos eventos recebidos.

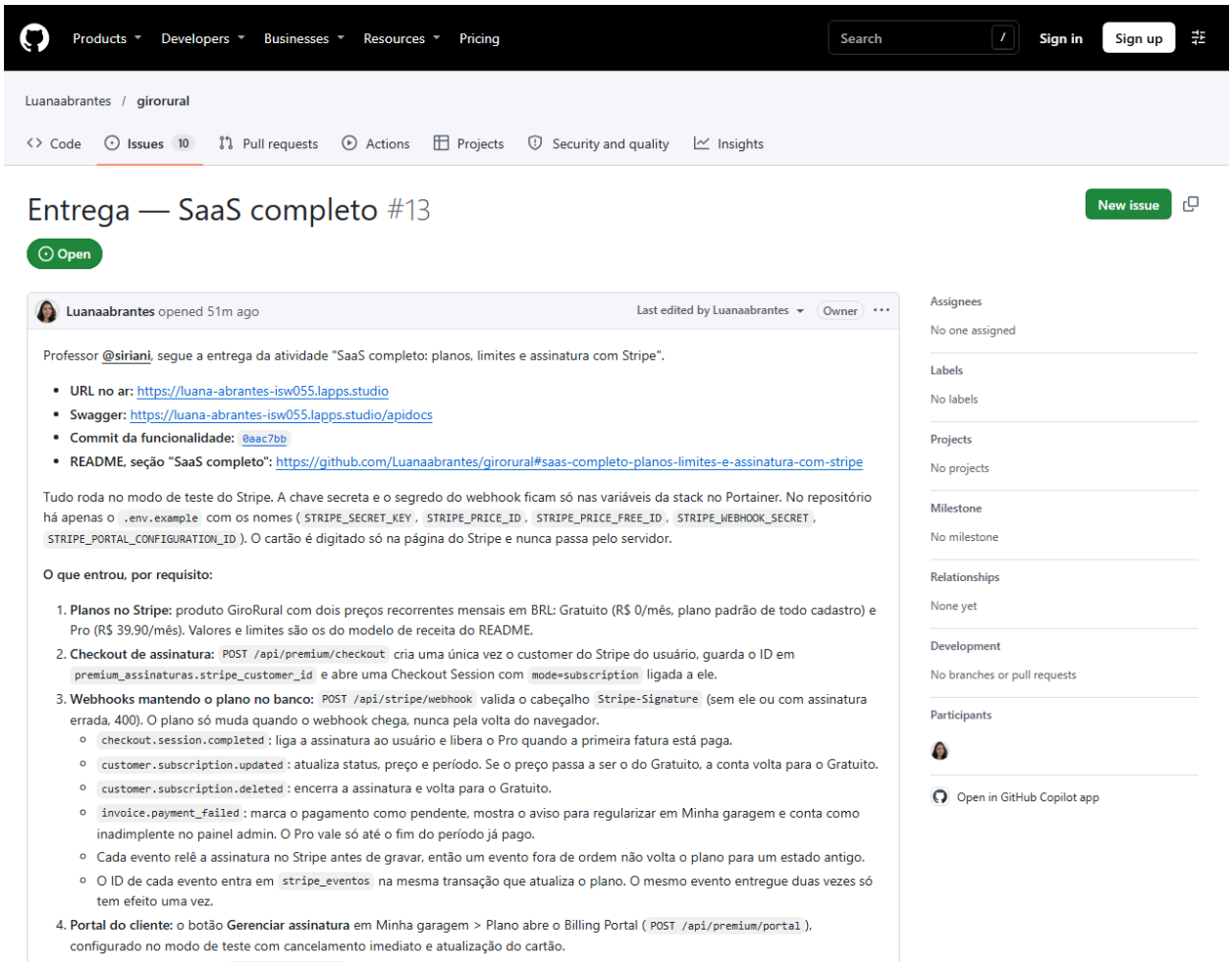


Figura 21: Atividade 11 — Issue de entrega 13 mencionando @siriani: página do GitHub e, abaixo, a API com “created_at”: “2026-11-13T02:14:24Z” (12/11/2026 23:14 em Brasília) (1/2: página do GitHub).

Estilos de formatação


```
{
  "url": "https://api.github.com/repos/Luanaabrantes/girorural/issues/13",
  "repository_url": "https://api.github.com/repos/Luanaabrantes/girorural",
  "labels_url": "https://api.github.com/repos/Luanaabrantes/girorural/issues/13/labels{/name}",
  "comments_url": "https://api.github.com/repos/Luanaabrantes/girorural/issues/13/comments",
  "events_url": "https://api.github.com/repos/Luanaabrantes/girorural/issues/13/events",
  "html_url": "https://github.com/Luanaabrantes/girorural/issues/13",
  "id": 6363229937,
  "node_id": "I_kwDOVVQZwc8AAAABe0cu8Q",
  "number": 13,
  "title": "Entrega — SaaS completo",
  "user": {
    "login": "Luanaabrantes",
    "id": 207479309,
    "node_id": "U_kgDODF3iDQ",
    "avatar_url": "https://avatars.githubusercontent.com/u/207479309?v=4",
    "gravatar_id": "",
    "url": "https://api.github.com/users/Luanaabrantes",
    "html_url": "https://github.com/Luanaabrantes",
    "followers_url": "https://api.github.com/users/Luanaabrantes/followers",
    "following_url": "https://api.github.com/users/Luanaabrantes/following{/other_user}",
    "gists_url": "https://api.github.com/users/Luanaabrantes/gists{/gist_id}",
    "starred_url": "https://api.github.com/users/Luanaabrantes/starred{/owner}{/repo}",
    "subscriptions_url": "https://api.github.com/users/Luanaabrantes/subscriptions",
    "organizations_url": "https://api.github.com/users/Luanaabrantes/orgs",
    "repos_url": "https://api.github.com/users/Luanaabrantes/repos",
    "events_url": "https://api.github.com/users/Luanaabrantes/events{/privacy}",
    "received_events_url": "https://api.github.com/users/Luanaabrantes/received_events",
    "type": "User",
    "user_view_type": "public",
    "site_admin": false
  },
  "labels": [

  ],
  "state": "open",
  "locked": false,
  "assignees": [

  ],
  "milestone": null,
  "comments": 0,
  "created_at": "2026-11-13T02:14:24Z",
  "updated_at": "2026-11-13T02:25:28Z",
  "closed_at": null,
  "assignee": null,
  "author_association": "OWNER",
  "active_lock_reason": null,
  "sub_issues_summary": {
    "total": 0,
    "completed": 0,
    "percent_completed": 0
  },
  "issue_dependencies_summary": {
    "blocked_by": 0,
    "total_blocked_by": 0,
    "blocking": 0,
    "total_blocking": 0
  },
  "body": "Professor @siriani, segue a entrega da atividade \"SaaS completo: planos, limites e assinatura com Stripe\".\n\n- **URL no ar:** https://luana-abrantes-
isw055.lapps.studio\n- **Swagger:** https://luana-abrantes-isw055.lapps.studio/apidocs\n- **Commit da funcionalidade:**
https://github.com/Luanaabrantes/girorural/commit/0aac7bbecb622f295d6cdd162c2129006539f611\n- **README, seção \"SaaS completo\":** https://github.com/Luanaabrantes/girorural#saas-
completo-planos-limites-e-assinatura-com-stripe\n\nTudo roda no modo de teste do Stripe. A chave secreta e o segredo do webhook ficam só nas variáveis da stack no Portainer. No
repositório há apenas o `.env.example` com os nomes (`STRIPE_SECRET_KEY`, `STRIPE_PRICE_ID`, `STRIPE_PRICE_FREE_ID`, `STRIPE_WEBHOOK_SECRET`, `STRIPE_PORTAL_CONFIGURATION_ID`). O
cartão é digitado só na página do Stripe e nunca passa pelo servidor.\n\n**O que entrou, por requisito:**\n\n1. **Planos no Stripe:** produto GiroRural com dois preços recorrentes
mensais em BRL: Gratuito (R$ 0/mês, plano padrão de todo cadastro) e Pro (R$ 39,90/mês). Valores e limites são os do modelo de receita do README.\n2. **Checkout de assinatura:**
`POST /api/premium/checkout` cria uma única vez o customer do Stripe do usuário, guarda o ID em `premium_assinaturas.stripe_customer_id` e abre uma Checkout Session com
`mode=subscription` ligada a ele.\n3. **Webhooks mantendo o plano no banco:** `POST /api/stripe/webhook` valida o cabeçalho `Stripe-Signature` (sem ele ou com assinatura errada,
400). O plano só muda quando o webhook chega, nunca pela volta do navegador.\n   - `checkout.session.completed`: liga a assinatura ao usuário e libera o Pro quando a primeira
fatura está paga.\n   - `customer.subscription.updated`: atualiza status, preço e período. Se o preço passa a ser o do Gratuito, a conta volta para o Gratuito.\n   -
`customer.subscription.deleted`: encerra a assinatura e volta para o Gratuito.\n   - `invoice.payment_failed`: marca o pagamento como pendente, mostra o aviso para regularizar em
Minha garagem e conta como inadimplente no painel admin. O Pro vale só até o fim do período já pago.\n   - Cada evento relê a assinatura no Stripe antes de gravar, então um evento
fora de ordem não volta o plano para um estado antigo.\n   - O ID de cada evento entra em `stripe_eventos` na mesma transação que atualiza o plano. O mesmo evento entregue duas
vezes só tem efeito uma vez.\n4. **Portal do cliente:** o botão **Gerenciar assinatura** em Minha garagem > Plano abre o Billing Portal (`POST /api/premium/portal`), configurado
no modo de teste com cancelamento imediato e atualização do cartão.\n5. **Limites por plano na API:** `POST /api/anuncios` lê o plano no banco a cada requisição. O Gratuito
permite 3 anúncios novos por mês. O 4º é recusado com 402 (`LIMITE_DO_PLANO`) e a mensagem \"Limite do plano Gratuito atingido: 3 anúncios por mês (você já criou 3 neste mês).
Assine o Pro por R$ 39,90/mês em Minha garagem > Plano para anunciar sem limite, ou aguarde o próximo mês.\"\n6. **Painel admin:** aba **Assinaturas e MRR** em `/admin.html` (`GET
/api/admin/assinaturas`, só `ADMIN`, 403 para os outros papéis): usuários por plano, assinaturas ativas, inadimplentes, MRR de teste (soma das assinaturas Pro com acesso válido) e
os últimos eventos do webhook.\n\nO CI roda 111 testes. Os de planos e assinatura ficam em `test/premium.test.js`, `test/saas-admin.test.js` e `test/rural-api.test.js`, e cobrem:
checkout concluído, evento repetido, checkout que não é de assinatura, pagamento recusado, cancelamento, troca Pro → Gratuito, portal, MRR e o 4º anúncio recusado no
Gratuito.\n\n**Prints, no cenário de uso do enunciado:**\n\nAssinatura do Pro paga com o cartão de teste 4242. No painel do Stripe (modo de teste), a assinatura GiroRural Pro
aparece ativa, com cobrança mensal de R$ 39,90:\n\n![Assinatura ativa no Stripe]
(https://raw.githubusercontent.com/Luanaabrantes/girorural/ec9022dbcbce71c985344a222793c89bbe7b9170/docs/entrega-saas/03-stripe-assinatura-ativa.jpg)\n\nWebhooks recebidos:
entregas no endpoint `/api/stripe/webhook` com 200 OK para `checkout.session.completed`, `customer.subscription.created` e `invoice.paid`:\n\n![Webhooks com 200 OK]
(https://raw.githubusercontent.com/Luanaabrantes/girorural/ec9022dbcbce71c985344a222793c89bbe7b9170/docs/entrega-saas/02-webhook-200.jpg)\n\nPainel admin, aba Assinaturas e MRR:
MRR de teste de R$ 39,90 (1 assinatura Pro ativa), usuários por plano, assinantes e os últimos eventos do webhook. Os e-mails de outros usuários foram borrados no print:\n\n!
[Painel admin com MRR](https://raw.githubusercontent.com/Luanaabrantes/girorural/ec9022dbcbce71c985344a222793c89bbe7b9170/docs/entrega-saas/04-admin-assinaturas-mrr.jpg)\n!
[Assinantes e últimos eventos](https://raw.githubusercontent.com/Luanaabrantes/girorural/ec9022dbcbce71c985344a222793c89bbe7b9170/docs/entrega-saas/05-admin-assinantes-
eventos.jpg)\n\nPortal do cliente aberto pelo botão \"Gerenciar assinatura\", com o cartão Visa •••• 4242 e a fatura de R$ 39,90 paga:\n\n[Portal do cliente](``
https://raw.githubusercontent.com/Luanaabrantes/girorural/ec9022dbcbce71c985344a222793c89bbe7b9170/docs/entrega-saas/06-portal-gerenciar-assinatura.jpg ``)\n\nCancelamento: depois
de cancelar no portal, o Stripe entrega o `customer.subscription.deleted` (200 OK) e a conta volta para o Gratuito:\n\n[Webhook customer.subscription.deleted](```
```


Figura 22: Atividade 11 — Issue de entrega 13 mencionando @siriani: página do GitHub e, abaixo, a API com “created_at”: “2026-11-13T02:14:24Z” (12/11/2026 23:14 em Brasília) (2/2: resposta da API do GitHub com a data e a hora em UTC).

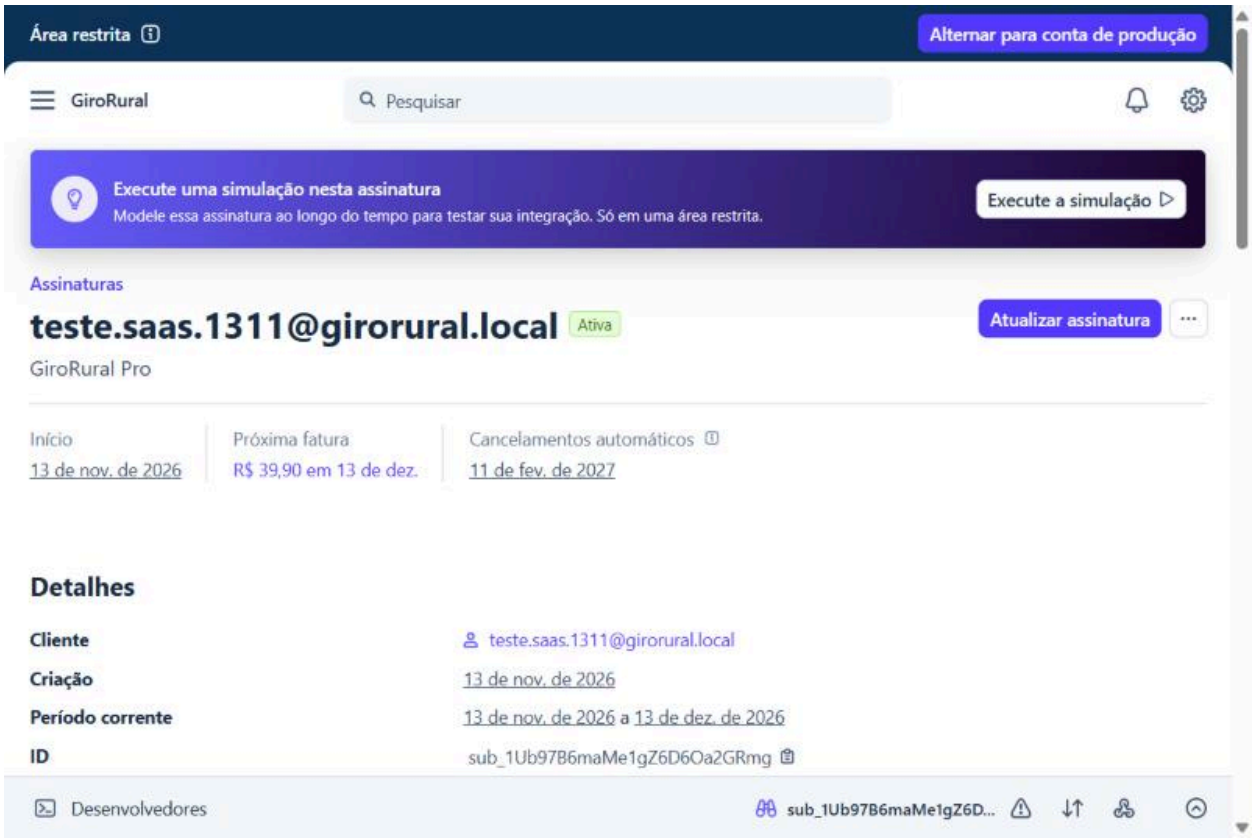


Figura 23: Atividade 11 — Assinatura GiroRural Pro ativa no Stripe (modo de teste).

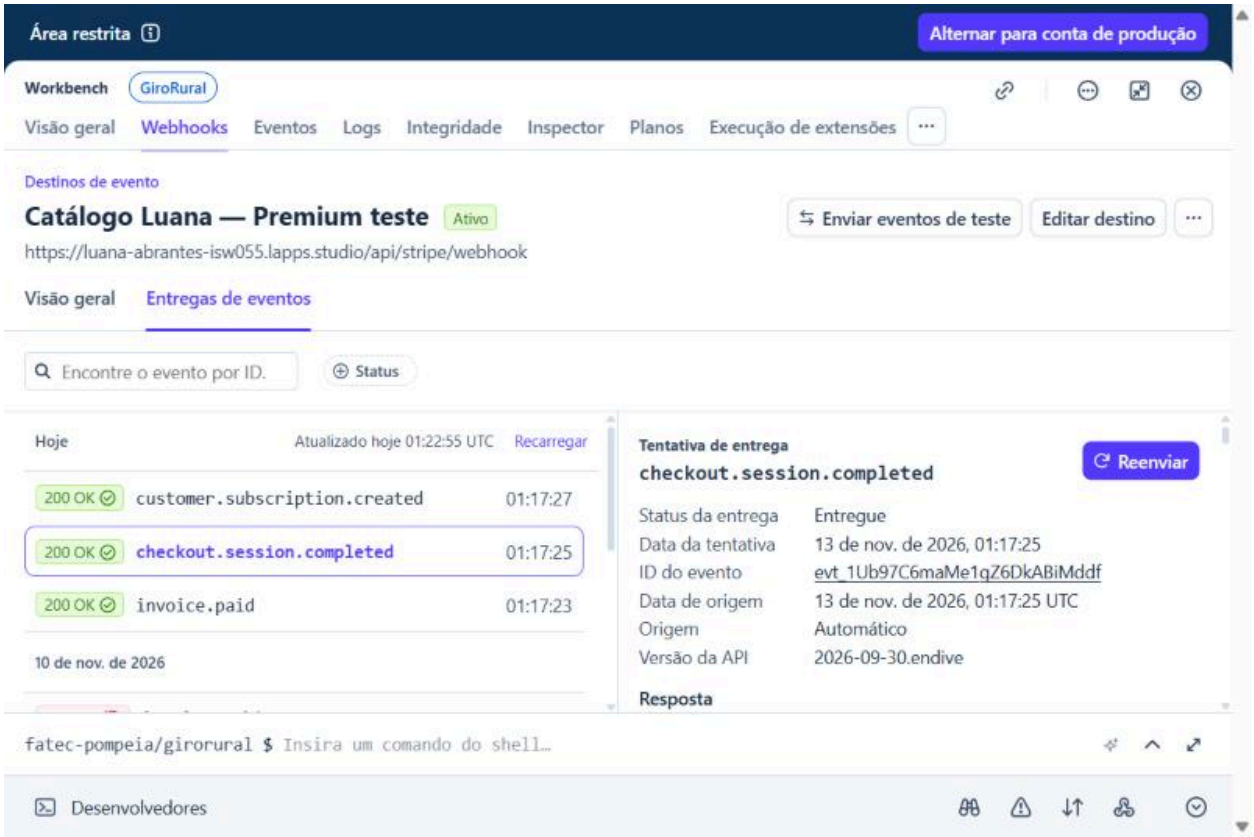


Figura 24: Atividade 11 — Webhooks entregues com resposta 200.

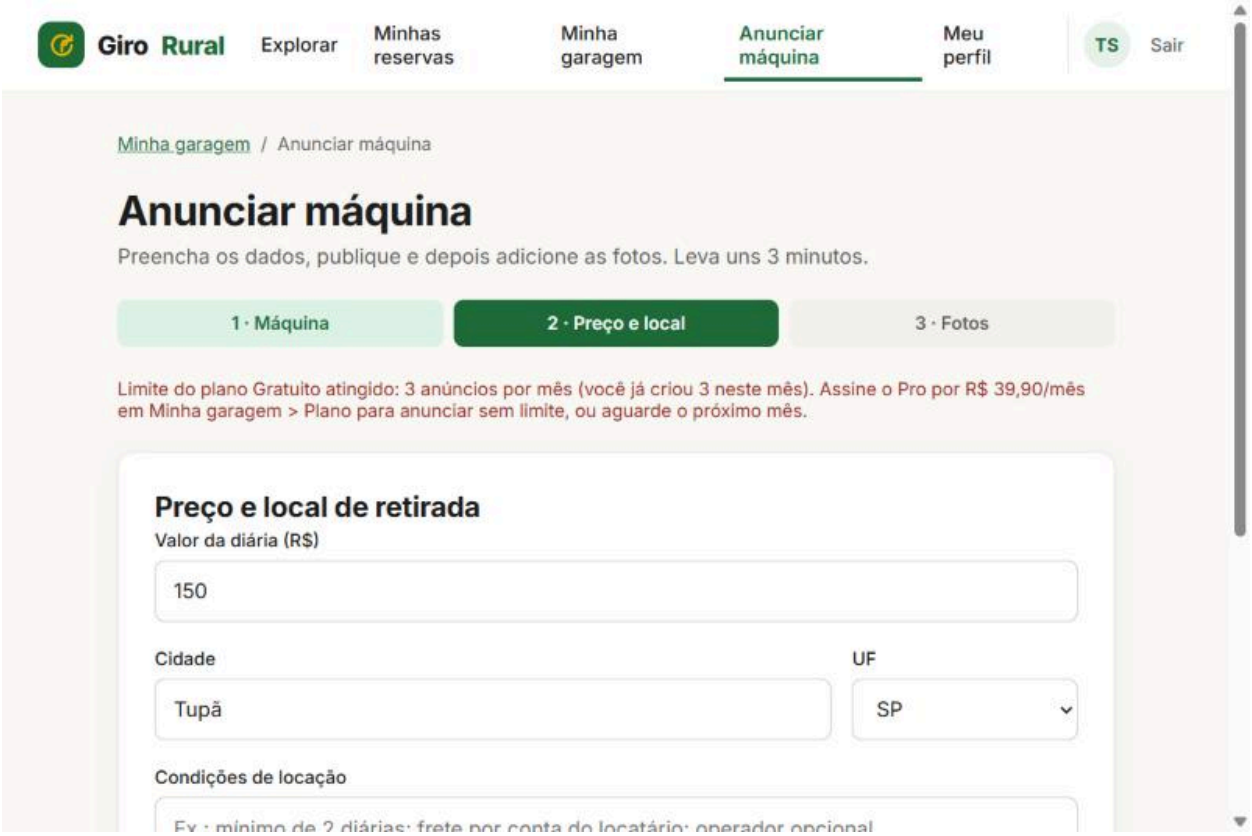


Figura 25: Atividade 11 — Plano Gratuito: a API recusa o 4º anúncio do mês com 402.

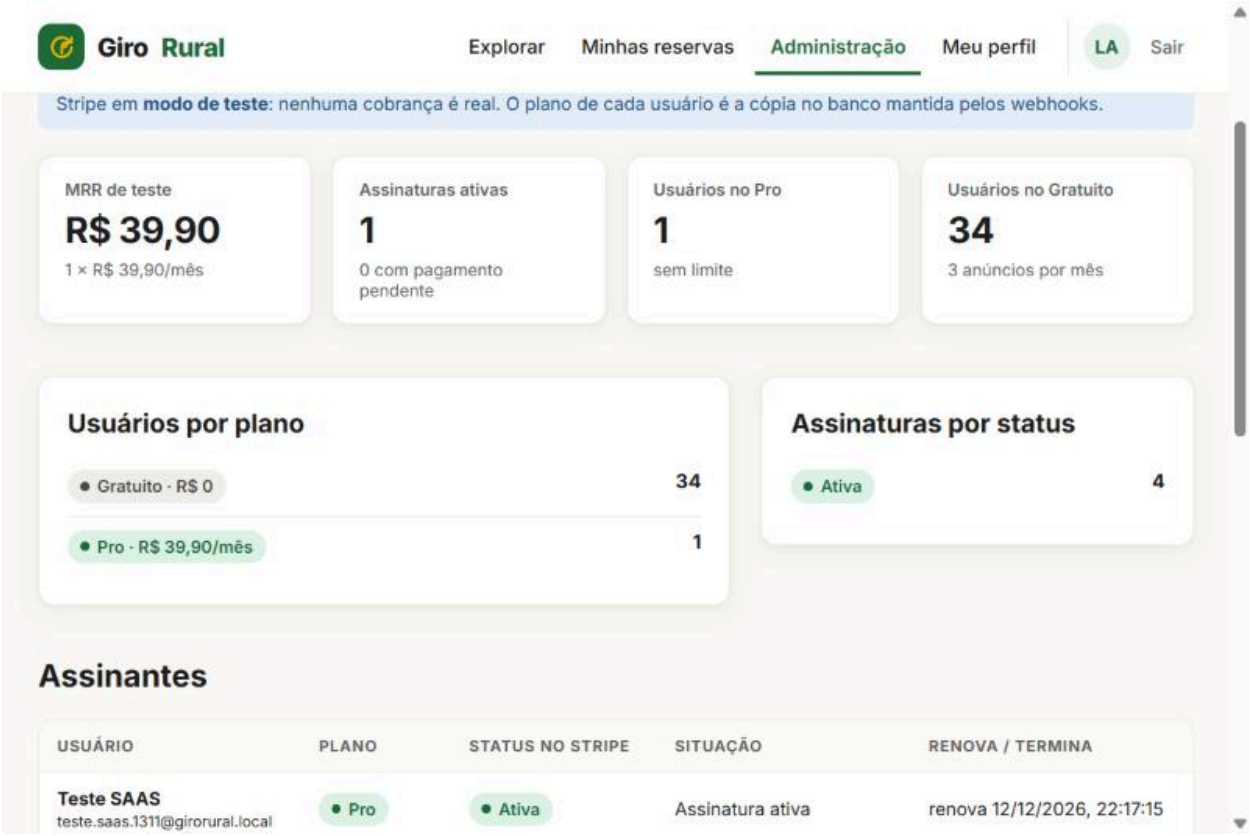


Figura 26: Atividade 11 — Painel admin com assinaturas e MRR de teste.

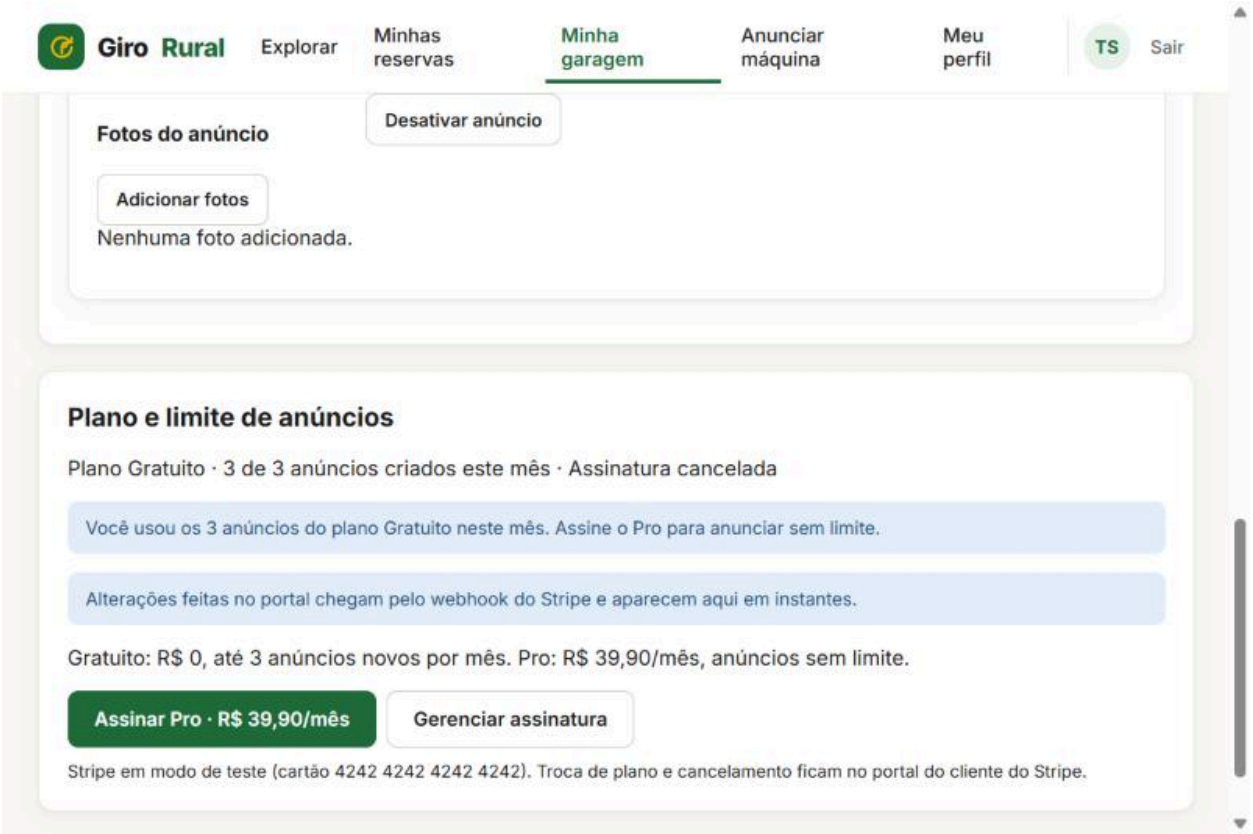


Figura 27: Atividade 11 — Cancelamento: webhook customer.subscription.deleted com 200 e conta de volta ao Gratuito.

Dificuldades e como foram resolvidas.

O ponto delicado foi garantir que só o webhook muda o plano e que um evento reenviado não tenha efeito duplicado; resolvi registrando o evento na mesma transação da atualização. Também foi preciso configurar o portal com cancelamento imediato para conseguir demonstrar a volta ao Gratuito no mesmo dia, e decidir que os anúncios do catálogo de demonstração não contam para o limite do plano.

6 Considerações finais

O maior aprendizado do bimestre foi ver as peças do primeiro bimestre funcionando como um sistema. Autenticação, papéis, auditoria, armazenamento de fotos e publicação na stack, que antes eram atividades separadas, passaram a sustentar um negócio com regras próprias. Quando precisei criar reservas, limitar anúncios por plano ou bloquear um locatário que tenta anunciar, eu já tinha onde apoiar cada regra, e isso mostrou na prática o valor de uma arquitetura em microsserviços bem separada.

A cobrança com o Stripe foi a parte que mais mudou a minha forma de pensar. Entendi que o navegador não é confiável para dizer que alguém pagou, que o webhook precisa ser validado e idempotente e que eventos podem chegar repetidos ou fora de ordem. A documentação reversa da atividade 8 e a seção de arquitetura deste relatório também me mostraram a diferença entre o que eu achava que tinha feito e o que o código realmente faz.

Para o GiroRural ser um produto de verdade ainda falta bastante. A cobrança está em modo de teste e o pagamento da locação na demonstração é simulado; para operar, seria preciso ativar a conta do Stripe e repassar o valor ao proprietário, por exemplo com Stripe Connect. Faltam também termos de uso e tratamento de dados conforme a LGPD, contrato e seguro da locação, avaliações entre usuários, backup automático do banco, alertas de monitoramento e deploy totalmente automático (hoje eu troco a tag

da imagem manualmente). Mesmo assim, termino o bimestre com um sistema no ar que cobra, limita e registra o que acontece, e com muito mais segurança para continuar.

7 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados.

Luana Beatriz Ribeiro Abrantes

Pompeia, 13/11/2026